TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH

VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ

BÁO CÁO TIỂU LUẬN

CHUYÊN ĐỀ HỆ THỐNG GIAO THÔNG THÔNG MINH

ĐỀ TÀI: XÂY DỰNG ỨNG DỤNG NHẬN DIỆN BIỂN SỐ XE

Ngành:	**Công Nghệ Thông Tin**
Học Phần:	**Chuyên Đề Hệ Thống Giao Thông Thông Minh**
Mã học phần:	**010112203814**
Giảng viên:	**Vũ Đình Long**
Sinh viên thực hiện:	**Nguyễn Phi Long**
	Chung Tiểu Phi
	Lê Minh Đức
	Lê Thanh Tuyền

Tp. Hồ Chí Minh, ngày 05 tháng 01 năm 2026

DANH SÁCH THÀNH VIÊN THỰC HIỆN

STT	Tên	Mssv
1	Nguyễn Phi Long	026205000232
2	Chung Tiểu Phi	096305005931
3	Lê Minh Đức	089205023742
4	Lê Thanh Tuyền	089305011455

MỤC LỤC

MỤC LỤC i
DANH MỤC HÌNH ẢNH v
DANH MỤC BẢNG BIỂU vii
DANH MỤC VIẾT TẮT viii
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI 1
1.1. Mục tiêu đề tài 1
1.1.1. Mục tiêu chính 1
1.1.2. Vấn đề cần giải quyết 1
1.1.3. Lĩnh vực nghiên cứu 1
1.1.4. Hạn chế 1
1.2. Phạm vi đề tài 2
1.2.1. Phạm vi đối tượng và ngữ cảnh ứng dụng 2
1.2.2. Phạm vi dữ liệu đầu vào 3
1.2.3. Phạm vi chức năng hệ thống 3
1.2.4. Phạm vi kỹ thuật và công nghệ 3
1.2.5. Phạm vi đánh giá và thử nghiệm 3
1.2.6. Phạm vi ngoài đề tài 3
1.2.7. Giới hạn và giả định 3
1.3. Roadmap và Timeline 4
1.3.1. Roadmap 4
1.3.2. Timeline 4
1.4. Kết quả đạt được 5
1.4.1. Xây dựng mô hình nhận diện biển số xe 5
1.4.2. Nhận dạng ký tự quang học trên biển số 5
1.4.3. Hoàn thiện pipeline xử lý end-to-end 5
1.4.4. Phát triển hệ thống quản lý và lưu trữ dữ liệu 5
1.4.5. Đánh giá hiệu năng hệ thống 5
1.4.6. Khả năng ứng dụng và mở rộng 6
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH YÊU CẦU 7

2.1. Giới thiệu chung về đề tài 7
2.2. Cơ sở lý thuyết 7
2.2.1. Thị giác máy tính (Computer Vision) trong hệ thống ALPR 7
2.2.2. Hệ thống cảm biến trong ITS 13
2.2.3. Giao tiếp dữ liệu trong hệ thống giao thông thông minh 14
2.2.4. Lưu trữ và quản lý dữ liệu 15
2.3. Phân tích yêu cầu hệ thống 15
2.3.1. Phạm vi và giới hạn của đề tài 16
2.3.2. Yêu cầu chức năng (Functional Requirements) 16
2.3.3. Yêu cầu phi chức năng (Non-Functional Requirements) 18
2.3.4. Các actor và kịch bản sử dụng (Actors & User Scenarios) 19
2.3.5. Thiết kế tổng quan hệ thống (High-level Design – tham chiếu) 19
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 20
3.1. Tổng quan kiến trúc hệ thống 20
3.2. Sơ đồ Use Case (Biểu đồ ca sử dụng) 20
3.3. Thiết kế kiến trúc và triển khai (Deployment Diagram) 21
3.4. Thiết kế cơ sở dữ liệu (Database Design) 22
3.5. Thiết kế luồng xử lý (Activity & Sequence Diagrams) 23
3.5.1. Quy trình xử lý xe vào/ra tự động 23
3.5.2. Biểu đồ tuần tự (Sequence Diagram) – API Call 24
3.6. Thiết kế luồng dữ liệu chi tiết (Data Flow Diagram - DFD) 24
3.7. Công nghệ sử dụng 25
3.8. Đặc tả các giao thức kết nối (API Endpoints) 26
3.9. Chi tiết thuật toán tiền xử lý ảnh 27
3.10. Cấu trúc chức năng Frontend 28
CHƯƠNG 4: HIỆN THỰC VÀ TRIỂN KHAI 29
4.1. Giới thiệu và quá trình hiện thực hóa 29
4.2. Công cụ sử dụng 29
4.2.1. Ngôn ngữ và thư viện lập trình 29
4.2.2. Giao tiếp dữ liệu và Môi trường 29
4.2.3. Bảng tóm tắt công cụ sử dụng 30
4.3. Mô tả các Module chính 30

4.3.1. Module Tiền xử lý & OCR 30
4.3.2. Module Hậu xử lý 31
4.3.3. Module quản lý xe vào/ra 32
4.3.4. Module quản lý dữ liệu 33
4.4. Screenshot/Result Images 34
4.5. How design → Code mapping 35
4.6. Flowchart thực tế 36
CHƯƠNG 5: KIỂM THỬ - ĐÁNH GIÁ 38
5.1. Giới thiệu về quá trình kiểm thử và đánh giá 38
5.2. Các trường hợp kiểm thử (test cases) 38
5.2.1. Tổng quan tập dữ liệu kiểm thử 38
5.2.2. Test cases theo điều kiện ánh sáng 38
5.2.3. Test cases theo mật độ phương tiện 39
5.2.4. Test cases occlusion và điều kiện bất lợi 40
5.3. Các chỉ số đánh giá (Metrics) 41
5.3.1. Metrics đánh giá mô hình phát hiện biển số (YOLOv8) 41
5.3.2. Metrics đánh giá mô hình OCR (EasyOCR) 45
5.3.3. Metrics đánh giá hệ thống end-to-end 47
5.4. Hạn chế của hệ thống 53
5.4.1. Độ chính xác và sai số mô hình 54
5.4.2. Hiệu năng tính toán (Performance) 54
5.5. Điều kiện triển khai thực tế 54
5.5.1. Yêu cầu về Hạ tầng phần cứng 54
5.5.2. Môi trường và Góc lắp đặt 54
5.6. Tiềm năng cải tiến 55
5.6.1. Tối ưu hóa mô hình và Thuật toán 55
5.6.2. Mở rộng chức năng hệ thống 55
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 56
6.1. Summary essence của hệ thống ALPR 56
6.1.1. Bản chất và phạm vi của hệ thống 56
6.1.2. Đặc trưng cốt lõi của hệ thống 56
6.1.3. Giá trị tổng quát và vai trò của hệ thống 56

6.2. Tổng quan và mục tiêu của dự án 57
6.2.1. Bối cảnh xuất phát của đề tài 57
6.2.2. Mục tiêu tổng thể của dự án 57
6.3. Tóm tắt cách tiếp cận và giải pháp đã thực hiện 57
6.3.1. Cách tiếp cận tổng thể 57
6.3.2. Các quyết định giải pháp chính 57
6.3.3. Định hướng triển khai 57
6.4. Tổng hợp các kết quả chính đạt được 58
6.4.1. Kết quả ở mức hệ thống 58
6.4.2. Đánh giá tổng thể 58
6.4.3. Giá trị thực nghiệm 58
6.5. Giá trị khoa học và đóng góp của đề tài 58
6.5.1. Đóng góp về mặt học thuật 58
6.5.2. Đóng góp về mặt thực tiễn 58
6.6. Ý nghĩa của hệ thống đối với Hệ thống Giao thông Thông minh (ITS) 59
6.7. Những hạn chế còn tồn tại 59
6.8. Hướng phát triển và mở rộng trong tương lai 59
6.9. Kết luận cuối cùng của đề tài 60

DANH MỤC HÌNH ẢNH

Hình 1.1: Roadmap project....4
Hình 1.2: Timeline project....4
Hình 2.1: Mô hình YOLO 7
Hình 2.2. label 1 10
Hình 2.3. label 2 10
Hình 2.4. label 3 11
Hình 2.5. label 4 11
Hình 2.6. label 5 12
Hình 2.7. label 6 12
Hình 3.1: Use Case Diagram 20
Hình 3.2: Deployment Diagram 21
Hình 3.3: Cấu trúc database design 22
Hình 3.4: Sơ đồ quy trình xử lý xe vào/ra tự động....23
Hình 3.5: Sơ đồ tuần tự (Sequence Diagram) 24
Hình 3.6: Sơ đồ DFD Level 0....24
Hình 3.7: Sơ đồ DFD Level 1....25
Hình 3.8: Thuật toán tiền xử lý ảnh....27
Hình 3.9: Cấu trúc Frontend....28
Hình 4.1: Hàm tiền xử lý ảnh sử dụng thuật toán CLAHE và Otsu Thresholding để tách nền. 30
Hình 4.2: Thuật toán hậu xử lý (Post-processing) sử dụng Regex và Dictionary Mapping để sửa lỗi nhận diện. 31
Hình 4.3: Hàm kiểm tra xem biển số này đã có trong bãi chưa 32
Hình 4.4: Dữ liệu phương tiện được lưu trữ thành công trong bảng MySQL. 33
Hình 4.5: kết nối DB. 33
Hình 4.6: Giao diện dữ liều ảnh đầu vào....34
Hình 4.7: Giao diện hiển thị thông tin xe vào và biển số đã nhận diện. 34
Hình 4.8: Nhật ký hoạt động (Log) của các Container Backend và Database trong quá trình vận hành. 35
Hình 4.9: Lưu đồ thực tế quy trình xử lý ảnh....36
Hình 4.10: Flowchart Nghiệp vụ Xe Vào/Ra (Tích hợp DB). 37
Hình 4.11: Sơ đồ hoạt động của hệ thống sau triển khai (Deployment Flow). 37

Hình 5.1: Test case trong điều kiện ánh sáng tốt 39
Hình 5.2: Test case trong điều kiện ánh sáng kém 39
Hình 5.3: Mật độ phương tiện thấp 40
Hình 5.4: Mật độ phương tiện cao.....40
Hình 5.5: Test cases occlusion.....41
Hình 5.6: F1-Confidence Curve (YOLO) 42
Hình 5.7: Precision-Confidence Curve (YOLO).....43
Hình 5.8: Precision-Recall Curve (YOLO) 44
Hình 5.9: Recall-Confidence Curve (YOLO) 45

DANH MỤC BẢNG BIỂU

Bảng 2.1: Bảng Functional Requirements xử lý nhận diện biển số 17
Bảng 2.2: Bảng Functional Requirements quản lý xe vào 17
Bảng 2.3: Bảng Functional Requirements quản lý xe ra 17
Bảng 2.4 : Bảng Functional Requirements quản lý dữ liệu và báo cáo 18
Bảng 2.5: Non-Functional Requirements 18
Bảng 3.1: Bảng tóm tắt các công nghệ và thư viện chính được sử dụng trong dự án...26
Bảng 3.2: Đặc tả các giao thức kết nối 26
Bảng 4.1: Danh mục công nghệ và công cụ triển khai hệ thống 30
Bảng 4.2: Ánh xạ từ Thiết kế (Chương 3) đến Hiện thực (Chương 4) 36
Bảng 5.1: Tham số training model 41
Bảng 5.2: chỉ số YOLOv8 41
Bảng 5.3: Định nghĩa các chỉ số đánh giá hệ thống 48
Bảng 5.4: Cấu hình suy luận mô hình EasyOCR trong quá trình đánh giá.....48
Bảng 5.5: Kết quả End-to-End Accuracy trên GPU 49
Bảng 5.6: Latency breakdown theo giai đoạn xử lý (GPU) 50
Bảng 5.7: Performance metrics tổng hợp (GPU) 50
Bảng 5.8: Kết quả End-to-End Accuracy trên CPU 51
Bảng 5.9: Latency breakdown theo giai đoạn xử lý (CPU) 52
Bảng 5.10: So sánh Character Error Rate (CER) trên GPU và CPU 52
Bảng 5.11: so sánh GPU vs CPU performance 52
Bảng 5.12: Performance metrics tổng hợp (CPU).....53
Bảng 5.13: GPU vs CPU Performance Comparison 53
Bảng 6.1: Tổng hợp kết quả chính đạt được.....58

DANH MỤC VIẾT TẮT

STT	Từ viết tắt	Ý nghĩa
1	ALPR	Automatic License Plate Recognition (Nhận diện biển số xe tự động)
2	API	Application Programming Interface (Giao diện lập trình ứng dụng)
3	CLAHE	Contrast Limited Adaptive Histogram Equalization (Cân bằng histogram thích ứng giới hạn tương phản)
4	CPU	Central Processing Unit (Đơn vị xử lý trung tâm)
5	CRUD	Create, Read, Update, Delete (Tạo, Đọc, Cập nhật, Xóa – các hoạt động cơ sở dữ liệu)
6	CV	Computer Vision (Thị giác máy tính)
7	DB	Database (Cơ sở dữ liệu)
8	FPS	Frames Per Second (Khung hình/giây)
9	GPU	Graphics Processing Unit (Đơn vị xử lý đồ họa)
10	HD	High Definition (Độ phân giải cao)
11	ITS	Intelligent Transportation Systems (Hệ thống giao thông thông minh)
12	JWT	JSON Web Token (Mã thông báo web JSON – dùng cho bảo mật)
13	LSTM	Long Short-Term Memory (Bộ nhớ ngắn hạn dài – mô hình học sâu)
14	MQTT	Message Queuing Telemetry Transport (Giao thức truyền tin nhắn hàng đợi đo lường)
15	OCR	Optical Character Recognition (Nhận dạng ký tự quang học)
16	ONNX	Open Neural Network Exchange (Trao đổi mạng nơ-ron mở)
17	PyTorch	(Thư viện học máy mở dựa trên Torch)
18	ResNet	Residual Network (Mạng dư – kiến trúc học sâu)
19	RTSP	Real-Time Streaming Protocol (Giao thức truyền phát thời gian thực)
20	SQL	Structured Query Language (Ngôn ngữ truy vấn có cấu trúc)
21	YOLOv8	You Only Look Once version 8 (Mô hình phát hiện đối tượng thời gian thực phiên bản 8)

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Mục tiêu đề tài

1.1.1. Mục tiêu chính

Đề tài "Xây dựng ứng dụng nhận diện biển số xe" hướng tới việc phát triển một hệ thống nhận diện biển số xe tự động (Automatic License Plate Recognition – ALPR) phục vụ cho các bài toán trong Hệ thống Giao thông Thông minh (ITS). Hệ thống có khả năng phát hiện phương tiện, trích xuất vùng biển số, nhận dạng ký tự và lưu trữ dữ liệu một cách chính xác, ổn định và có khả năng mở rộng trong thực tế.

Mục tiêu tổng quát của đề tài là xây dựng một pipeline xử lý end-to-end, kết hợp các mô hình học sâu hiện đại nhằm đáp ứng yêu cầu về độ chính xác, tốc độ xử lý và khả năng ứng dụng trong các kịch bản giao thông thực tế như bãi giữ xe thông minh, kiểm soát ra/vào, thu phí tự động hoặc giám sát giao thông.

1.1.2. Vấn đề cần giải quyết

Mặc dù các hệ thống nhận diện biển số xe đã được nghiên cứu và triển khai rộng rãi, việc áp dụng trong bối cảnh giao thông tại Việt Nam vẫn còn gặp nhiều thách thức. Đề tài tập trung giải quyết các vấn đề chính sau:

- Biển số xe xuất hiện với kích thước nhỏ, góc nhìn nghiêng hoặc bị che khuất một phần trong ảnh và video.
- Điều kiện môi trường phức tạp như ánh sáng yếu, ngược sáng, mưa hoặc nhiễu hình ảnh làm giảm độ chính xác nhận diện.
- Sự đa dạng về kiểu chữ, khoảng cách ký tự và chất lượng in trên biển số gây khó khăn cho quá trình nhận dạng ký tự.
- Việc tích hợp các mô hình phát hiện và nhận dạng thành một hệ thống hoàn chỉnh, hoạt động ổn định trong thời gian thực.

Những vấn đề trên đòi hỏi một giải pháp tổng thể, kết hợp hiệu quả giữa mô hình học sâu, xử lý ảnh và thiết kế hệ thống.

1.1.3.Lĩnh vực nghiên cứu

Đề tài thuộc lĩnh vực Hệ thống Giao thông Thông minh (Intelligent Transportation Systems – ITS), đồng thời giao thoa với các lĩnh vực nghiên cứu sau:

- Thị giác máy tính (Computer Vision).
- Học sâu và trí tuệ nhân tạo (Deep Learning, Artificial Intelligence).
- Xử lý ảnh số (Digital Image Processing).
- Hệ thống thông tin và cơ sở dữ liệu.

Thông qua việc kết hợp các lĩnh vực trên, đề tài hướng đến xây dựng một hệ thống ALPR có tính ứng dụng thực tiễn trong giao thông thông minh.

1.1.4.Hạn chế

Bên cạnh các mục tiêu và định hướng rõ ràng, đề tài cũng tồn tại một số hạn chế nhất định:

- Dữ liệu huấn luyện còn hạn chế về số lượng và độ đa dạng so với các hệ thống thương mại.
- Hệ thống chưa được kiểm thử trên quy mô lớn hoặc trong điều kiện giao thông thực tế liên tục.
- Chưa xem xét sâu các vấn đề về bảo mật dữ liệu và quyền riêng tư.
- Hiệu năng xử lý thời gian thực còn phụ thuộc vào cấu hình phần cứng.

1.2. Phạm vi đề tài

Nhằm đảm bảo đề tài được triển khai đúng trọng tâm, khả thi trong thời gian môn học và phù hợp với định hướng của lĩnh vực Giao thông Thông minh (ITS), phạm vi nghiên cứu của đề tài được xác định rõ ràng theo các khía cạnh sau.

1.2.1. Phạm vi đối tượng và ngữ cảnh ứng dụng

- Đối tượng nghiên cứu chính là biển số xe cơ giới đường bộ, bao gồm ô tô và xe máy.
- Ngữ cảnh ứng dụng tập trung vào các kịch bản ITS phổ biến như:
  + Hệ thống bãi đỗ xe thông minh (Smart Parking).
  + Kiểm soát phương tiện ra/vào khu vực (cổng cơ quan, khu dân cư, bãi giữ xe).
  + Thu thập dữ liệu phục vụ giám sát và thống kê giao thông.
- Hệ thống được xây dựng với giả định camera đặt cố định, góc quay tương đối ổn định.

1.2.2. Phạm vi dữ liệu đầu vào

- Dữ liệu đầu vào bao gồm:
  + Ảnh tĩnh chụp phương tiện từ camera.
  + Video ngắn mô phỏng luồng xe ra/vào.
- Dữ liệu được thu thập từ:
  + Nguồn dữ liệu tự xây dựng.
  + Một số bộ dữ liệu công khai phù hợp với bài toán nhận diện biển số.
- Biển số trong dữ liệu chủ yếu là biển số theo chuẩn Việt Nam.

1.2.3. Phạm vi chức năng hệ thống

Hệ thống tập trung triển khai các chức năng cốt lõi sau:

- Phát hiện phương tiện và xác định vị trí biển số trong ảnh/video.
- Trích xuất vùng ảnh chứa biển số.
- Nhận dạng chuỗi ký tự trên biển số.
- Lưu trữ thông tin nhận diện (biển số, thời gian, hình ảnh) vào cơ sở dữ liệu.
- Hỗ trợ truy vấn và hiển thị kết quả phục vụ mục đích giám sát và báo cáo.

1.2.4. Phạm vi kỹ thuật và công nghệ

- Áp dụng các mô hình học sâu trong thị giác máy tính:
  + YOLOv8 cho bài toán phát hiện biển số.
  + EasyOCR cho bài toán nhận dạng ký tự.
- Ngôn ngữ và công cụ triển khai chính: Python, OpenCV, MySQL.
- Hệ thống được triển khai và thử nghiệm trên môi trường máy tính cá nhân hoặc máy chủ.

1.2.5. Phạm vi đánh giá và thử nghiệm

- Đánh giá hệ thống dựa trên:
  + Độ chính xác phát hiện và nhận dạng biển số.
  + Thời gian suy luận (Inference Time).
  + Độ ổn định khi xử lý liên tục nhiều khung hình.
- Thử nghiệm trên cả CPU và GPU để so sánh hiệu năng.

1.2.6. Phạm vi ngoài đề tài

Các nội dung sau không nằm trong phạm vi nghiên cứu của đề tài:

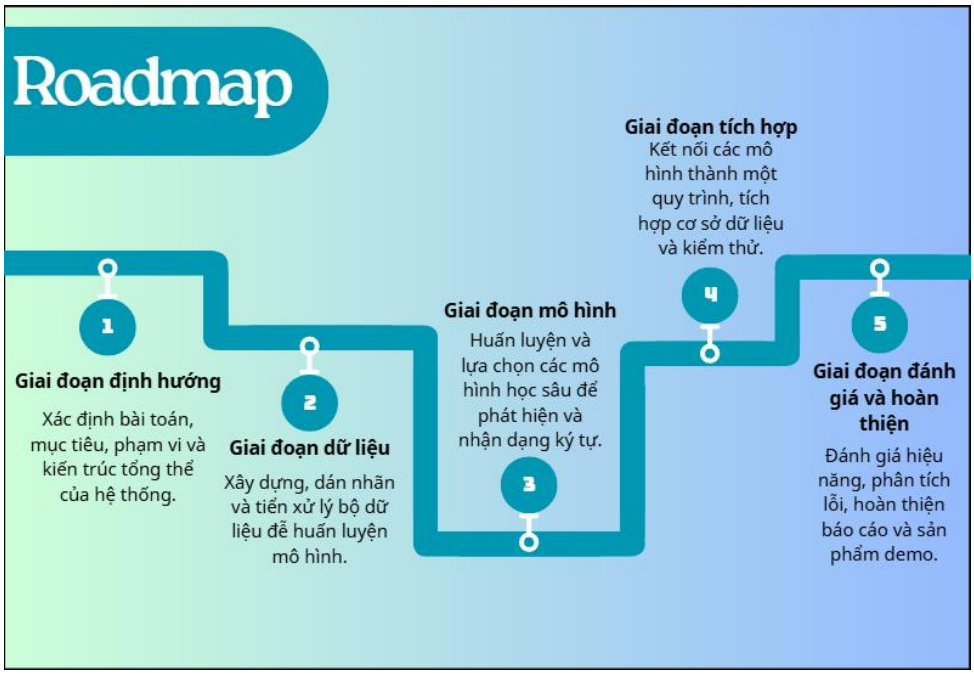
- Nhận diện khuôn mặt hoặc danh tính người điều khiển phương tiện.
- Phân tích hành vi giao thông nâng cao hoặc dự báo ùn tắc.
- Triển khai hệ thống trên hạ tầng giao thông đô thị quy mô lớn.
- Các vấn đề pháp lý, bảo mật dữ liệu cá nhân và quyền riêng tư.

1.2.7. Giới hạn và giả định

- Chất lượng nhận diện phụ thuộc vào độ phân giải camera và điều kiện môi trường.
- Dữ liệu huấn luyện còn giới hạn về số lượng và độ đa dạng.
- Hệ thống ưu tiên mục tiêu nghiên cứu và minh họa kỹ thuật hơn là thương mại hóa.

1.3. Roadmap và Timeline

1.3.1. Roadmap



Hình 1.1: Roadmap project

1.3.2. Timeline

Hình 1.2: Timeline project

1.4. Kết quả đạt được

Sau quá trình triển khai theo roadmap và timeline đã đề ra, đề tài đã đạt được các kết quả chính sau đây, phản ánh cả khía cạnh kỹ thuật, tính hoàn chỉnh của hệ thống và khả năng ứng dụng trong lĩnh vực ITS.

1.4.1. Xây dựng mô hình nhận diện biển số xe

Đề tài đã xây dựng và huấn luyện thành công mô hình phát hiện biển số xe dựa trên YOLOv8. Mô hình có khả năng xác định chính xác vị trí biển số trong ảnh và video, đáp ứng tốt các tình huống phổ biến như:

- Nhiều phương tiện xuất hiện đồng thời trong khung hình.
- Biển số có kích thước khác nhau do thay đổi khoảng cách và góc nhìn.
- Điều kiện ánh sáng thay đổi ở mức độ cho phép.

Kết quả cho thấy mô hình có độ ổn định cao trong môi trường thử nghiệm và phù hợp cho các bài toán ITS quy mô nhỏ và trung bình.

1.4.2. Nhận dạng ký tự quang học trên biển số

Mô hình EasyOCR được huấn luyện và tinh chỉnh để nhận dạng chuỗi ký tự trên vùng biển số đã được trích xuất. Hệ thống có khả năng:

- Nhận dạng chính xác các ký tự chữ và số trên biển số.
- Giảm thiểu nhầm lẫn giữa các ký tự có hình dạng tương đồng.
- Xuất kết quả dưới dạng chuỗi ký tự phục vụ cho việc lưu trữ và truy vấn.

Kết quả nhận dạng đạt độ chính xác phù hợp với mục tiêu nghiên cứu và demo của đề tài.

1.4.3. Hoàn thiện pipeline xử lý end-to-end

Một pipeline xử lý hoàn chỉnh đã được xây dựng, bao gồm các bước: thu nhận dữ liệu, tiền xử lý ảnh, phát hiện biển số, nhận dạng ký tự và hậu xử lý kết quả. Pipeline này cho phép hệ thống:

- Hoạt động tự động từ dữ liệu đầu vào đến kết quả đầu ra.
- Dễ dàng mở rộng hoặc thay thế mô hình trong các giai đoạn sau.
- Đảm bảo tính nhất quán trong quá trình xử lý dữ liệu.

1.4.4. Phát triển hệ thống quản lý và lưu trữ dữ liệu

Hệ thống đã được tích hợp với cơ sở dữ liệu MySQL để lưu trữ các thông tin quan trọng, bao gồm:

- Biển số xe đã nhận dạng.
- Thời điểm phương tiện xuất hiện.
- Hình ảnh gốc và hình ảnh biển số đã trích xuất.

Việc lưu trữ dữ liệu có cấu trúc giúp hỗ trợ truy vấn, thống kê và phục vụ cho các chức năng báo cáo trong các hệ thống ITS.

1.4.5. Đánh giá hiệu năng hệ thống

Đề tài đã tiến hành đánh giá hiệu năng hệ thống trên cả CPU và GPU, tập trung vào các tiêu chí:

- Độ chính xác phát hiện và nhận dạng.
- Thời gian suy luận (Inference Time).
- Độ ổn định khi xử lý liên tục nhiều khung hình.

Kết quả đánh giá cho thấy hệ thống đáp ứng tốt yêu cầu demo và nghiên cứu trong phạm vi đề tài.

1.4.6. Khả năng ứng dụng và mở rộng

Hệ thống nhận diện biển số xe được xây dựng trong đề tài có khả năng ứng dụng vào nhiều kịch bản ITS như:

- Quản lý bãi đỗ xe thông minh.
- Kiểm soát phương tiện ra/vào khu vực.
- Thu thập dữ liệu phục vụ thống kê giao thông.

Ngoài ra, kiến trúc hệ thống cho phép mở rộng thêm các chức năng nâng cao trong tương lai như xử lý thời gian thực, triển khai trên thiết bị edge hoặc tích hợp với các hệ thống ITS khác.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHÂN TÍCH YÊU CẦU

2.1. Giới thiệu chung về đề tài

Đề tài "Xây dựng ứng dụng nhận diện biển số xe" thuộc chuyên đề Hệ thống giao thông thông minh (Intelligent Transportation Systems - ITS), tập trung phát triển hệ thống nhận diện biển số xe tự động (Automatic License Plate Recognition - ALPR) dành riêng cho biển số xe Việt Nam. Hệ thống sử dụng các kỹ thuật học sâu (Deep Learning) kết hợp thị giác máy tính (Computer Vision) để tự động phát hiện vị trí biển số xe trong ảnh và nhận dạng các ký tự trên biển số, chuyển đổi thành văn bản số hóa.

Hệ thống bao gồm hai giai đoạn chính:

- Nhận diện vị trí biển số xe (License Plate Detection) sử dụng mô hình YOLOv8.
- Nhận dạng ký tự quang học (Optical Character Recognition - OCR) sử dụng mô hình EasyOCR.

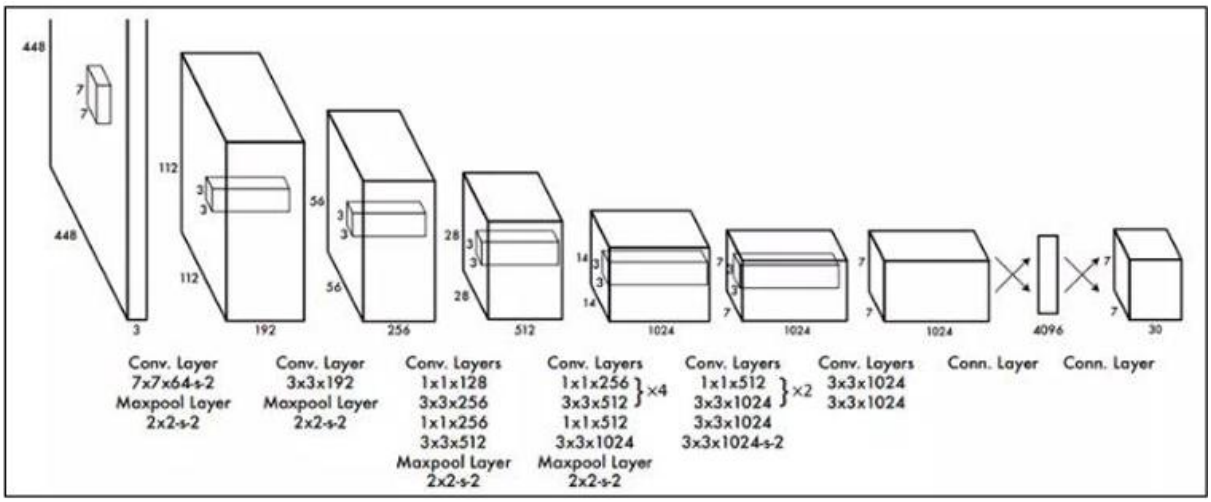
2.2. Cơ sở lý thuyết

2.2.1. Thị giác máy tính (Computer Vision) trong hệ thống ALPR

Thị giác máy tính là nền tảng cốt lõi cho hệ thống ALPR, cho phép máy tính "nhìn" và hiểu nội dung hình ảnh từ camera. Trong đề tài, các kỹ thuật thị giác máy tính được áp dụng để giải quyết hai bài toán chính: phát hiện đối tượng (object detection) và nhận dạng văn bản (text recognition).

2.2.1.1. Nhận diện đối tượng – Mô hình YOLOv8

YOLOv8 (You Only Look Once version 8) là mô hình học sâu thuộc nhóm Object Detection, do Ultralytics phát triển. Đây là phiên bản mới nhất kế thừa và cải tiến từ YOLOv5, mang lại độ chính xác và tốc độ xử lý vượt trội.



Hình 2.1: Mô hình YOLO

Nguyên lý hoạt động của YOLO:

- Ảnh đầu vào được chia thành lưới (grid) có kích thước S × S (ví dụ: 7×7 ở YOLOv1, sau này linh hoạt hơn).

- Mỗi ô lưới chịu trách nhiệm dự đoán B bounding box (khung bao quanh đối tượng) cùng với confidence score (độ tin cậy rằng ô lưới chứa đối tượng và độ chính xác vị trí).
- Mỗi bounding box bao gồm: tọa độ trung tâm (x, y), chiều rộng và cao (w, h) – tất cả được chuẩn hóa theo kích thước ảnh.
- Đồng thời dự đoán xác suất lớp (class probabilities) cho C lớp đối tượng.
- Confidence score = Pr(Object) × IoU(pred, truth), giúp loại bỏ các dự đoán kém.

Cải tiến ở YOLOv8: YOLOv8 do Ultralytics phát triển là phiên bản cải tiến đáng kể so với YOLOv5 và các phiên bản trước:

- Anchor-free detection head.
- Backbone và neck hiện đại hơn.
- Tốc độ và độ chính xác cân bằng.
- Dễ sử dụng và tích hợp.
- Hiệu suất vượt trội.

Lý do lựa chọn YOLOv8 cho bài toán phát hiện biển số xe Việt Nam:

- Tốc độ xử lý cao, phù hợp ứng dụng thời gian thực.
- Độ chính xác vượt.
- Dễ huấn luyện lại (fine-tune) trên tập dữ liệu tùy chỉnh.
- Hỗ trợ anchor-free giúp định vị chính xác hơn với biển số có kích thước và tỷ lệ thay đổi lớn.
- Trong đề tài, YOLOv8 đạt mAP@0.5-0.95 = 0.995, Precision = Recall = 1.0 trên tập validation.

2.2.1.2. Nhận dạng ký tự quang học – Mô hình EasyOCR

EasyOCR là một thư viện mã nguồn mở phổ biến trong cộng đồng thị giác máy tính, được xây dựng trên nền tảng framework PyTorch. Khác với các mô hình OCR truyền thống, EasyOCR sử dụng sự kết hợp giữa Deep Learning hiện đại để đạt được độ chính xác cao và tốc độ xử lý nhanh, hỗ trợ đa ngôn ngữ (bao gồm cả tiếng Việt).

Cấu trúc mô hình: Mô hình hoạt động dựa trên một quy trình (pipeline) gồm hai thành phần chính:

- Detector (CRAFT): Sử dụng thuật toán CRAFT (Character Region Awareness for Text Detection) để phát hiện các vùng có chứa văn bản trong ảnh. CRAFT không chỉ tìm các khung hình chữ nhật mà còn có thể phát hiện văn bản theo các đường cong hoặc bị biến dạng.
- Recognizer (CRNN): Sau khi vùng văn bản được trích xuất, EasyOCR sử dụng mạng nơ-ron tích chập (CNN) để trích xuất các đặc trưng hình thái của ký tự, kết hợp với mạng LSTM (Long Short-Term Memory) để nhận diện chuỗi ký tự dựa trên ngữ cảnh và thứ tự. Cuối cùng, lớp CTC (Connectionist Temporal Classification) được dùng để giải mã kết quả văn bản cuối cùng.

Ưu điểm khi áp dụng cho biển số xe:

- Gọn nhẹ và Hiệu quả: EasyOCR có kích thước mô hình tối ưu, cho phép chạy tốt trên cả CPU và GPU, phù hợp cho việc triển khai các hệ thống quản lý bãi xe thông minh thực tế.
- Hỗ trợ đa ngôn ngữ: Khả năng nhận diện tốt các ký tự in hoa và chữ số theo bảng chữ cái Latinh, rất phù hợp với định dạng biển số xe tại Việt Nam.
- Khả năng chống nhiễu: Nhờ module CRAFT, EasyOCR có thể tách biệt tốt phần biển số khỏi các nhiễu môi trường xung quanh (như logo hãng xe, ốc vít hoặc bụi bẩn).
- Dễ dàng tích hợp: Cung cấp API đơn giản cho Python, giúp rút ngắn thời gian phát triển và hiện thực hóa các module hậu xử lý như lọc Regex (Regular Expression) cho biển số.

2.2.1.3. Tiền xử lý và hậu xử lý ảnh

Các bước tiền xử lý:

- Resize ảnh về kích thước chuẩn
- Normalize: Chuẩn hóa giá trị pixel về khoảng [0,1] hoặc mean/std của tập huấn luyện.
- Enhancement: Tăng độ tương phản, điều chỉnh độ sáng/sắc nét để cải thiện chất lượng ảnh kém.
- Noise reduction: Giảm nhiễu Gaussian, giúp mô hình tập trung vào đặc trưng chính.

Các bước hậu xử lý:

- Chuẩn hóa định dạng biển số theo quy chuẩn Việt Nam (ví dụ “51H 12345”).
- Sửa lỗi OCR phổ biến: đối chiếu quy tắc (ký tự chữ/số đúng vị trí), sửa nhầm lẫn tương đồng
- Validate kết quả: loại bỏ chuỗi không hợp lệ, tăng độ tin cậy đầu ra.

2.2.1.4. Tăng cường dữ liệu (Data Augmentation)

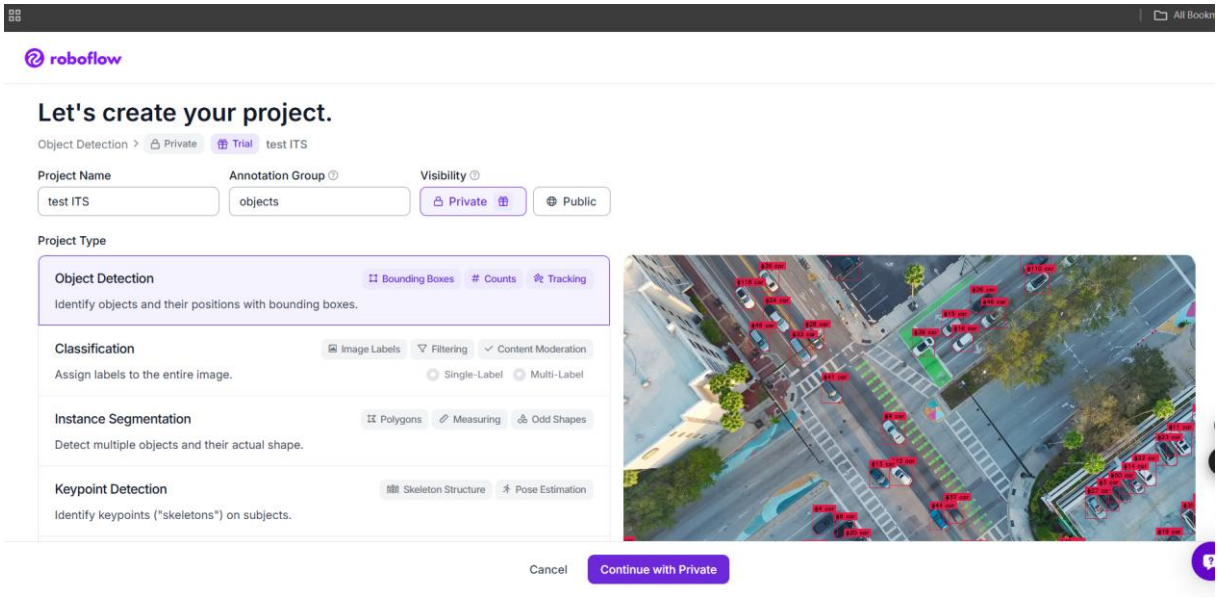
Các kỹ thuật được áp dụng: Rotation ±10°, Brightness ±20%, Gaussian Noise, Horizontal Flip, Crop & Zoom.

Mục đích và lợi ích:

- Tăng độ robust của mô hình với điều kiện thực tế đa dạng (ánh sáng, góc chụp, chất lượng ảnh).
- Giảm overfitting: mô hình học được các đặc trưng tổng quát thay vì memorize dữ liệu huấn luyện.
- Cải thiện generalization trên tập validation và test.

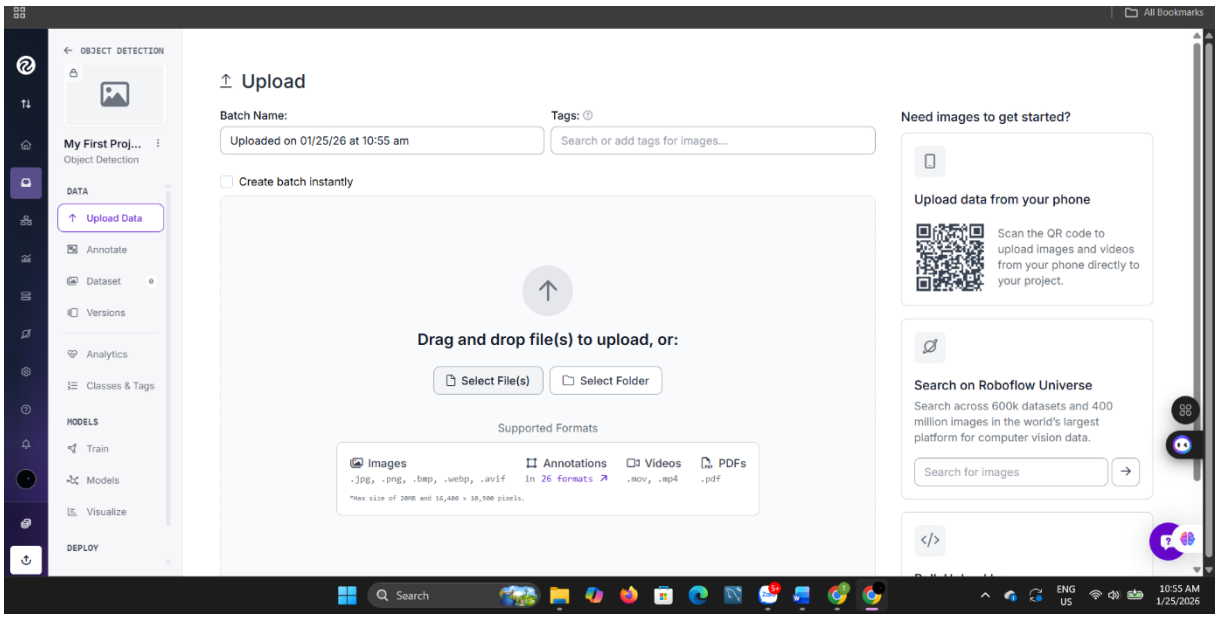
2.2.1.5. Dán nhãn dữ liệu bằng Roboflow

- Tạo dự án trên Roboflow:
  + Đặt loại dự án là Object Detection.
  + Định nghĩa lớp nhãn (class) “license_plate”.

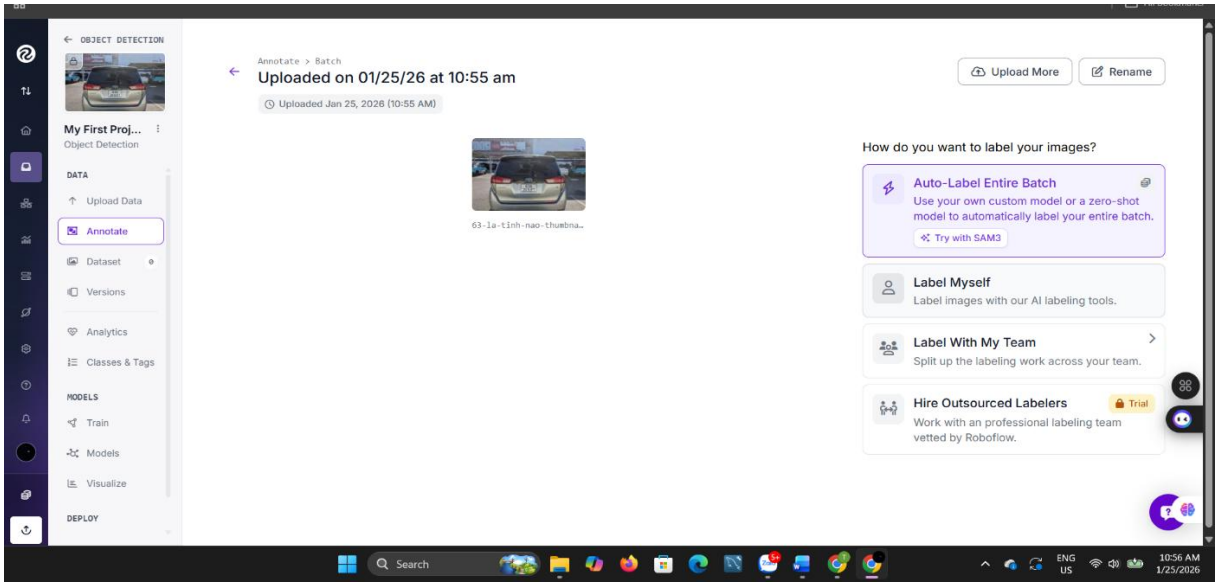


Hình 2.2. label 1

- Tải lên tập ảnh gốc:
  + Nhập các ảnh biển số xe từ tập dữ liệu gốc (Kaggle).
  + Có thể tải hàng loạt ảnh (.jpg, .png).

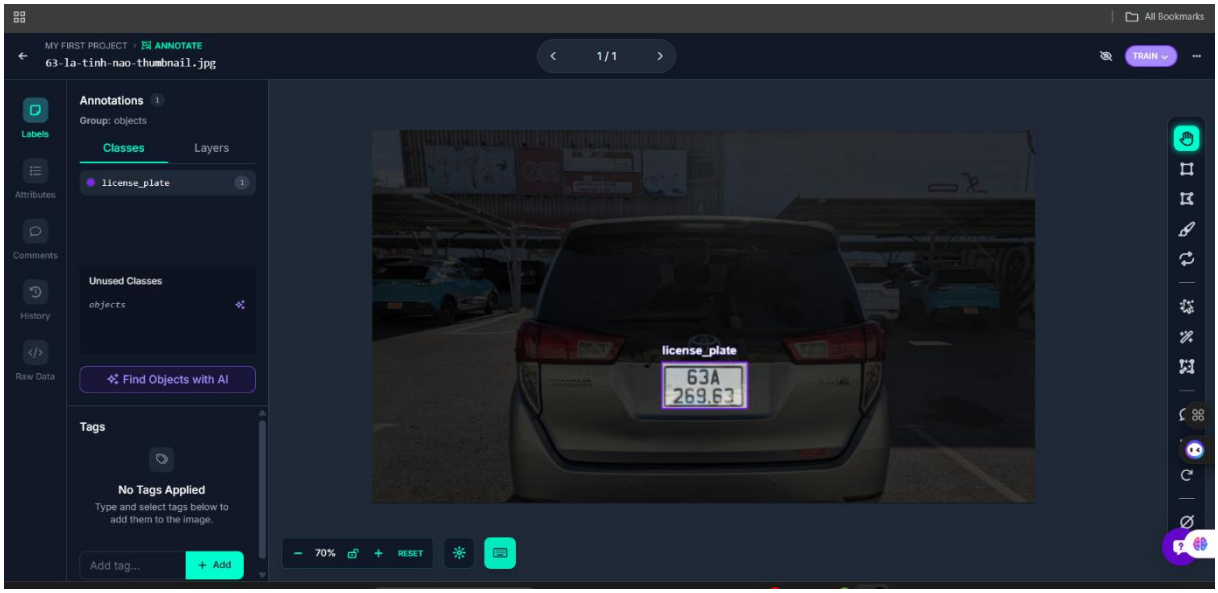


Hình 2.3. label 2



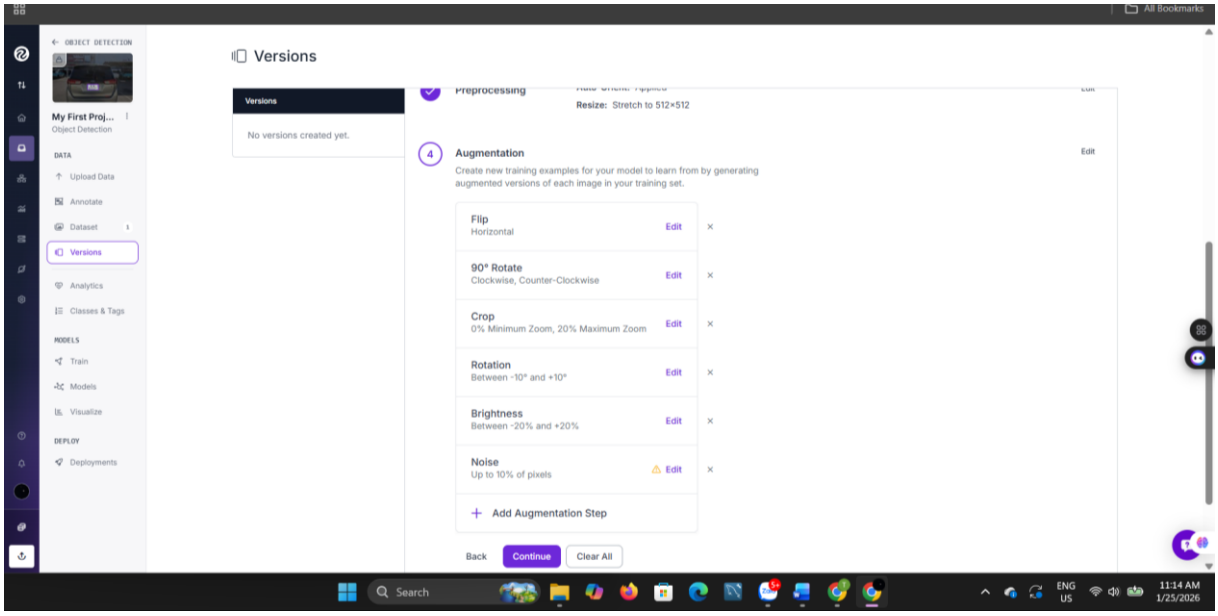
Hình 2.4. label 3

- Dán nhãn thủ công:
  + Sử dụng công cụ bounding box để bao quanh biển số xe trong ảnh.
  + Gán nhãn tương ứng "license_plate" cho từng vùng.



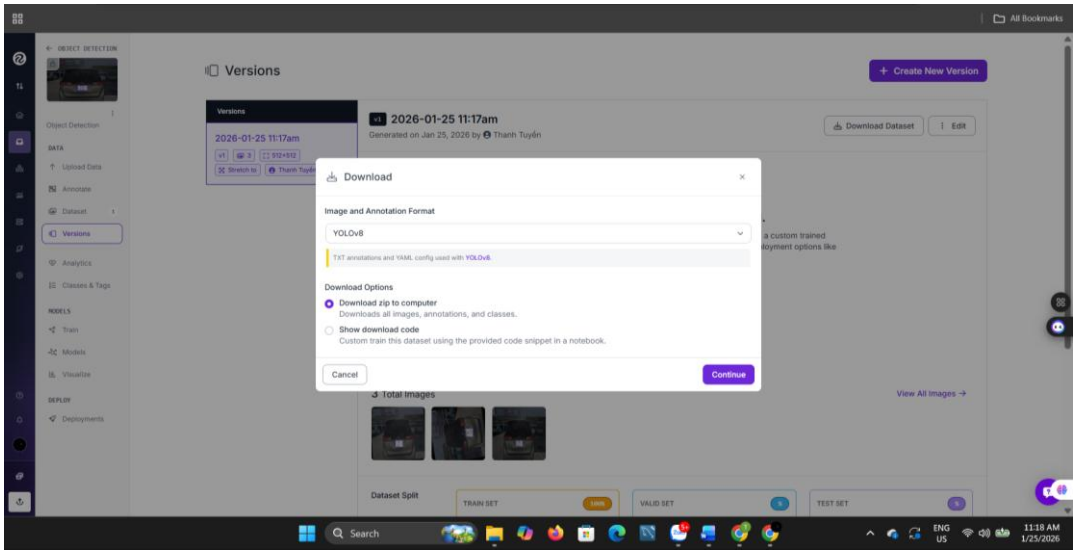
Hình 2.5. label 4

- Áp dụng augmentation trực tiếp trên Roboflow:
  + Chọn các kỹ thuật như xoay, lật, điều chỉnh độ sáng.
  + Roboflow tự động tạo các phiên bản mở rộng của ảnh.



Hình 2.6. label 5

- Xuất dữ liệu:
  + Chọn định dạng là YOLOv8 (Ultralytics).
  + Tải về tập dữ liệu đã dán nhãn gồm ba thư mục: train, valid, test cùng file cấu hình data.yaml (chứa đường dẫn, số lớp, tên lớp).



Hình 2.7. label 6

Ưu điểm: Giao diện trực quan, quản lý phiên bản dataset, tự động định dạng tương thích, tiết kiệm thời gian so với dán nhãn thủ công thuần túy.

2.2.1.6. Các chỉ số đánh giá mô hình

Đối với YOLOv8:

- mAP@0.5: Độ chính xác trung bình ở ngưỡng IoU = 0.5.
- mAP@0.5:0.95: Trung bình qua nhiều ngưỡng IoU (chỉ số nghiêm ngặt hơn).
- Precision: Tỷ lệ dự đoán đúng trong các dự đoán dương tính.

- Recall: Tỷ lệ đối tượng thật được phát hiện.
- Inference Time: Thời gian xử lý một ảnh (ms).

Đối với EasyOCR:

- Character Error Rate (CER): Tỷ lệ ký tự sai so với ground truth (Min CER = 4.74%).
- Character Accuracy: Tỷ lệ ký tự đúng (≈95.26%).

Các đường cong đặc:

- F1-Confidence Curve: Đánh giá sự ổn định theo ngưỡng confidence.
- Precision-Confidence, Recall-Confidence, Precision-Recall Curve.
- Loss curves: Box Loss, Classification Loss, DFL Loss (YOLO); Training/Validation Loss (EasyOCR).

Các chỉ số và đường cong được sử dụng để chứng minh mô hình ổn định, hội tụ tốt và không có overfitting nghiêm trọng (đặc biệt YOLOv8).

2.2.2. Hệ thống cảm biến trong ITS

Trong hệ thống giao thông thông minh (Intelligent Transportation Systems – ITS), hệ thống cảm biến là thành phần nền tảng đảm nhiệm chức năng thu thập dữ liệu từ môi trường giao thông thực tế. Đối với bài toán nhận diện biển số xe tự động (Automatic License Plate Recognition – ALPR), chất lượng và độ tin cậy của dữ liệu cảm biến có ảnh hưởng trực tiếp đến hiệu quả của các thuật toán xử lý ảnh và nhận dạng ký tự. Do đó, việc lựa chọn và tích hợp cảm biến phù hợp là yêu cầu mang tính quyết định trong quá trình xây dựng hệ thống.

2.2.2.1. Camera – Cảm biến chính cho ALPR

Camera là cảm biến cốt lõi trong hệ thống ALPR, có nhiệm vụ thu nhận hình ảnh phương tiện và biển số trong điều kiện giao thông động. Hiệu năng của hệ thống nhận diện phụ thuộc lớn vào các đặc tính kỹ thuật và điều kiện khai thác của camera.

2.2.2.1.1. Yêu cầu về độ phân giải, điều kiện ánh sáng và góc chụp:

- Độ phân giải: Camera sử dụng trong hệ thống ALPR cần có độ phân giải đủ cao (tối thiểu Full HD – 1920×1080) để đảm bảo vùng biển số chiếm đủ số điểm ảnh cần thiết cho quá trình phát hiện và nhận dạng ký tự. Độ phân giải càng cao thì khả năng trích xuất chi tiết ký tự càng chính xác.
- Điều kiện ánh sáng: Camera cần được trang bị các công nghệ hỗ trợ như Wide Dynamic Range (WDR), High Dynamic Range (HDR) hoặc hồng ngoại (IR) nhằm đảm bảo khả năng hoạt động ổn định trong điều kiện ánh sáng yếu, ban đêm hoặc môi trường có độ tương phản cao.
- Góc chụp: Góc quan sát của camera cần được thiết kế sao cho mặt phẳng biển số gần vuông góc với trục quang học của ống kính. Việc hạn chế góc nghiêng giúp giảm biến dạng phối cảnh và nâng cao độ chính xác của thuật toán nhận diện ký tự quang học (OCR).

2.2.2.1.2. Các hạn chế thực tế:

Trong môi trường giao thông thực tế, camera phải đối mặt với nhiều yếu tố gây suy giảm chất lượng hình ảnh:

- Motion blur: Xảy ra khi phương tiện di chuyển với tốc độ cao trong khi tốc độ màn trập của camera không đủ lớn, dẫn đến hiện tượng mờ chuyển động.
- Backlight: Ánh sáng nền mạnh hoặc ánh sáng phản xạ từ bề mặt xe có thể làm biển số bị lóa, gây mất thông tin ký tự.
- Thời tiết khắc nghiệt: Các điều kiện như mưa lớn, sương mù, bụi bẩn hoặc ánh sáng không ổn định làm giảm độ tương phản và độ sắc nét của hình ảnh, ảnh hưởng đến hiệu quả nhận diện.

2.2.2.2. Các cảm biến bổ trợ khác (mở rộng)

Để nâng cao độ chính xác và độ tin cậy của hệ thống ALPR trong ITS, các cảm biến bổ trợ thường được tích hợp nhằm cung cấp thêm thông tin hỗ trợ cho camera.

2.2.2.2.1. Radar

Radar sử dụng sóng điện từ để phát hiện và đo các tham số chuyển động của phương tiện như khoảng cách và vận tốc:

- Hoạt động ổn định trong điều kiện thời tiết xấu và không phụ thuộc vào ánh sáng.
- Hỗ trợ xác định thời điểm phương tiện đi vào vùng giám sát, từ đó kích hoạt camera và hệ thống xử lý ALPR.
- Giảm số lượng dữ liệu hình ảnh không cần thiết, góp phần tối ưu tài nguyên xử lý.

2.2.2.2.2. LiDAR

LiDAR (Light Detection and Ranging) sử dụng tia laser để thu thập dữ liệu không gian ba chiều:

- Cung cấp thông tin chính xác về vị trí, hình dạng và khoảng cách của phương tiện.
- Hỗ trợ phân tách nhiều phương tiện trong cùng một khu vực quan sát, đặc biệt tại các nút giao thông phức tạp.
- Tuy nhiên, chi phí triển khai cao và yêu cầu kỹ thuật phức tạp khiến LiDAR thường chỉ được sử dụng trong các hệ thống ITS quy mô lớn hoặc các nghiên cứu thử nghiệm.

2.2.3. Giao tiếp dữ liệu trong hệ thống giao thông thông minh

Trong ITS, giao tiếp dữ liệu là thành phần đảm bảo sự liên kết giữa các cảm biến, hệ thống xử lý và trung tâm quản lý. Đối với ứng dụng ALPR, dữ liệu cần được truyền tải với độ trễ thấp, độ tin cậy cao và khả năng mở rộng linh hoạt.

2.2.3.1. MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông nhẹ, được sử dụng rộng rãi trong các hệ thống IoT và ITS:

- Hoạt động theo mô hình Publish/Subscribe, giúp giảm sự phụ thuộc trực tiếp giữa thiết bị gửi và thiết bị nhận.
- Phù hợp với hệ thống ALPR nhờ khả năng truyền dữ liệu thời gian thực với băng thông thấp.
- Hỗ trợ mở rộng hệ thống khi số lượng camera và thiết bị tăng lên.

2.2.3.2. V2X (Vehicle-to-Everything)

V2X là tập hợp các công nghệ cho phép phương tiện giao tiếp với các thành phần xung quanh:

- V2V (Vehicle-to-Vehicle) và V2I (Vehicle-to-Infrastructure) cho phép chia sẻ thông tin giao thông theo thời gian thực.
- Trong hệ thống ALPR, V2X hỗ trợ tích hợp dữ liệu nhận diện biển số với các hệ thống điều khiển giao thông, góp phần nâng cao an toàn và hiệu quả vận hành.

2.2.4. Lưu trữ và quản lý dữ liệu

Hệ thống ALPR tạo ra khối lượng lớn dữ liệu hình ảnh và dữ liệu nhận diện, do đó yêu cầu một cơ chế lưu trữ và quản lý dữ liệu hiệu quả.

2.2.4.1. Hệ quản trị cơ sở dữ liệu MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ được lựa chọn phổ biến nhờ:

- Tính ổn định và khả năng xử lý dữ liệu lớn.
- Dễ triển khai và tích hợp với các ứng dụng web và hệ thống ITS.
- Hỗ trợ các cơ chế bảo mật và phân quyền truy cập.

2.2.4.2. Cấu trúc các bảng dữ liệu chính

- vehicles: Lưu trữ thông tin phương tiện như biển số, loại xe và các thuộc tính liên quan.
- detection_logs: Ghi nhận lịch sử nhận diện gồm thời gian, vị trí và độ tin cậy của kết quả.
- images: Lưu trữ hình ảnh hoặc đường dẫn hình ảnh phục vụ việc kiểm tra và đối soát.
- parking: Quản lý thông tin ra vào bãi đỗ, thời gian lưu trú và trạng thái phương tiện.

Thiết kế cơ sở dữ liệu hợp lý giúp hệ thống ALPR hoạt động ổn định, dễ mở rộng và đáp ứng yêu cầu của các ứng dụng giao thông thông minh.

2.3. Phân tích yêu cầu hệ thống

Phần này phân tích yêu cầu hệ thống ALPR để hỗ trợ quản lý bãi đỗ xe trong ITS, xác định phạm vi, chức năng, phi chức năng, actors, kịch bản sử dụng và thiết kế tổng quan.

2.3.1. Phạm vi và giới hạn của đề tài

Đề tài tập trung vào việc xây dựng ứng dụng nhận diện biển số xe (ALPR - Automatic License Plate Recognition) nhằm hỗ trợ quản lý bãi đỗ xe cơ bản trong hệ thống giao thông thông minh. Phạm vi của đề tài được giới hạn như sau:

- Phạm vi về dữ liệu: Hệ thống sử dụng bộ dữ liệu gồm 5135 ảnh biển số xe Việt Nam, tập trung vào hình ảnh tĩnh (không xử lý video thời gian thực). Dữ liệu bao gồm các loại biển số một dòng và hai dòng theo quy chuẩn Việt Nam, được thu thập từ các nguồn công khai và dán nhãn thủ công.
- Phạm vi về kỹ thuật: Ứng dụng sử dụng mô hình YOLOv8 cho việc phát hiện biển số và EasyOCR cho nhận dạng ký tự quang học. Toàn bộ hệ thống được triển khai bằng ngôn ngữ lập trình Python, với quá trình huấn luyện mô hình thực hiện trên nền tảng Kaggle với tài nguyên GPU T4 x2 để tối ưu hóa hiệu suất tính toán.
- Phạm vi về chức năng: Hệ thống bao gồm pipeline ALPR chính (phát hiện và nhận dạng biển số) kết hợp với quản lý bãi đỗ xe cơ bản, bao gồm ghi nhận xe vào/ra, lưu trữ dữ liệu và báo cáo đơn giản. Điều này nhằm hỗ trợ tự động hóa quy trình quản lý giao thông tại các bãi đỗ xe.

Các giới hạn và chức năng nằm ngoài phạm vi đề tài bao gồm:

- Không xử lý video thời gian thực (real-time video streaming) hoặc theo dõi chuyển động (tracking) của xe.
- Không tích hợp các chức năng nâng cao như thanh toán tự động, nhận diện loại xe (ví dụ: xe máy, ô tô), màu sắc xe, hoặc tích hợp với hệ thống camera động.
- Không mở rộng đến các ứng dụng khác trong ITS như giám sát giao thông đô thị toàn diện hoặc tích hợp với cảm biến khác (ngoài camera cơ bản).

Việc giới hạn này giúp tập trung nguồn lực vào việc phát triển và đánh giá hiệu quả của pipeline ALPR, đồng thời đảm bảo tính khả thi trong khuôn khổ tiểu luận.

2.3.2. Yêu cầu chức năng (Functional Requirements)

Yêu cầu chức năng mô tả các tính năng cụ thể mà hệ thống phải thực hiện để đáp ứng mục tiêu. Các chức năng chính được phân loại như sau, với từng chức năng được trình bày dưới dạng bảng để dễ theo dõi.

2.3.2.1. Chức năng xử lý nhận diện biển số (ALPR Pipeline)

Thành phần	Mô tả
Phát hiện biển số	Sử dụng YOLOv8 để xác định vị trí biển số trong ảnh, xử lý góc chụp và ánh sáng biến đổi.
Nhận dạng ký tự	Sử dụng EasyOCR để trích xuất ký tự, hỗ trợ định dạng Việt Nam (ví dụ: 12A-345.67 hoặc 12-AB-34567).
Tiền xử lý ảnh	Resize, normalize, enhancement, noise reduction để cải thiện chất lượng đầu vào.

Hậu xử lý	Chuẩn hóa định dạng biển số, sửa lỗi OCR dựa trên quy tắc Việt Nam (loại bỏ ký tự không hợp lệ).
Yêu cầu hiệu suất	Độ chính xác cao (>95%), thời gian xử lý <1 giây/ảnh để hỗ trợ ứng dụng thực tế.

Bảng 2.1: Bảng Functional Requirements xử lý nhận diện biển số

2.3.2.2. Chức năng quản lý xe vào (Vehicle Entry)

Thành phần	Mô tả
Chụp ảnh	Chụp ảnh xe và biển số qua camera khi xe vào bãi đỗ.
Xử lý ALPR	Thực hiện pipeline ALPR để lấy thông tin biển số.
Lưu trữ dữ liệu	Lưu thời gian vào, vị trí đỗ (nếu có) vào cơ sở dữ liệu.
Kiểm tra và cập nhật	Kiểm tra biển số tồn tại (xe đã đỗ trước đó), cập nhật trạng thái "Đang đỗ".
Thông báo	Gửi thông báo hoặc hiển thị cho người quản lý qua giao diện web/ứng dụng.
Lợi ích	Tự động hóa quy trình nhập xe, giảm thời gian thủ công.

Bảng 2.2: Bảng Functional Requirements quản lý xe vào

2.3.2.3. Chức năng quản lý xe ra (Vehicle Exit)

Thành phần	Mô tả
Chụp ảnh	Chụp ảnh xe và biển số để xác nhận khi xe rời bãi đỗ.
Xử lý ALPR	Thực hiện pipeline ALPR để xác nhận biển số.
Tính toán	So sánh với dữ liệu xe vào, tính thời gian đỗ và phí đỗ (công thức cơ bản).
Cập nhật dữ liệu	Cập nhật trạng thái "Đã ra" trong cơ sở dữ liệu, xóa thông tin tạm thời nếu cần.
Hiển thị	Hiển thị thông tin cho người dùng, bao gồm thời gian đỗ và xác nhận thanh toán thủ công.
Lợi ích	Hỗ trợ kiểm soát dòng xe ra, tránh tình trạng ùn tắc.

Bảng 2.3: Bảng Functional Requirements quản lý xe ra

2.3.2.4. Chức năng quản lý dữ liệu và báo cáo

Thành phần	Mô tả
Lưu trữ dữ liệu	Lưu vào MySQL với bảng: vehicles (thông tin xe), detection_logs (lịch sử nhận diện), images (ảnh lưu trữ), parking (quản lý bãi đỗ).
Tra cứu	Cho phép tìm kiếm thông tin theo biển số hoặc thời gian.

Báo cáo thống kê	Tạo báo cáo như số lượng xe vào/ra theo ngày, danh sách xe đang đỗ, lịch sử lỗi nhận diện.
Quản lý dữ liệu	Sao lưu định kỳ và xóa dữ liệu cũ để tối ưu hóa lưu trữ.
Lợi ích	Giúp người quản lý giám sát hệ thống một cách hiệu quả.

Bảng 2.4 : Bảng Functional Requirements quản lý dữ liệu và báo cáo

2.3.3. Yêu cầu phi chức năng (Non-Functional Requirements)

Yêu cầu phi chức năng tập trung vào các thuộc tính chất lượng của hệ thống, đảm bảo tính thực tiễn và bền vững.

Yêu cầu	Mô tả
Hiệu suất (Performance)	Hệ thống phải xử lý ít nhất 10 ảnh/giây trên GPU T4, với độ chính xác nhận diện trên 95% cho dữ liệu kiểm tra. Thời gian phản hồi cho mỗi yêu cầu quản lý xe dưới 2 giây.
Khả năng sử dụng (Usability)	Giao diện người dùng đơn giản, thân thiện, hỗ trợ trên web hoặc ứng dụng di động. Hướng dẫn sử dụng rõ ràng, không yêu cầu kiến thức chuyên sâu về AI.
Độ tin cậy (Reliability)	Hệ thống phải hoạt động ổn định 24/7, với tỷ lệ lỗi nhận diện dưới 5%. Tích hợp cơ chế xử lý lỗi (error handling) cho các trường hợp ảnh mờ hoặc ánh sáng kém.
Khả năng duy trì (Maintainability)	Mã nguồn được tổ chức modular, dễ cập nhật mô hình (ví dụ: retrain YOLOv8 với dữ liệu mới). Sử dụng tài liệu code và version control (như Git).
Bảo mật (Security)	Bảo vệ dữ liệu biển số bằng mã hóa (encryption), kiểm soát truy cập (authentication) cho người quản lý. Tuân thủ quy định bảo mật dữ liệu cá nhân theo luật Việt Nam.
Hiệu năng mở rộng (Scalability)	Hệ thống có thể mở rộng để xử lý nhiều camera đồng thời, với khả năng tích hợp cloud computing nếu cần tăng tải.
Tương thích (Compatibility)	Hỗ trợ các định dạng ảnh phổ biến (JPEG, PNG), chạy trên các nền tảng Windows/Linux, và tích hợp với camera thông thường.

Bảng 2.5: Non-Functional Requirements

2.3.4. Các actor và kịch bản sử dụng (Actors & User Scenarios)

Danh sách các actor chính:

- Người quản lý bãi đỗ (Administrator): Quản lý hệ thống, xem báo cáo, cập nhật dữ liệu.
- Hệ thống camera (System Sensor): Tự động chụp ảnh và gửi dữ liệu cho ALPR.
- Người lái xe (Driver): Tương tác gián tiếp qua việc đỗ xe, không trực tiếp sử dụng hệ thống.
- Quản trị viên cơ sở dữ liệu (DB Admin): Quản lý lưu trữ và sao lưu dữ liệu.

Các kịch bản sử dụng điển hình:

- Xe vào: Camera chụp ảnh, ALPR xử lý biển số, lưu thời gian vào, cập nhật trạng thái.
- Xe ra: Camera chụp ảnh, so sánh biển số, tính thời gian đỗ, cập nhật trạng thái và báo cáo phí.
- Tra cứu: Người quản lý nhập biển số để xem lịch sử vào/ra.
- Thống kê: Tạo báo cáo hàng ngày về số lượng xe, thời gian đỗ trung bình.

Các kịch bản này được thiết kế để minh họa luồng sử dụng thực tế, đảm bảo hệ thống hỗ trợ hiệu quả cho quản lý giao thông.

2.3.5. Thiết kế tổng quan hệ thống (High-level Design – tham chiếu)

Thiết kế tổng quan cung cấp cái nhìn tổng thể về hệ thống, bao gồm các sơ đồ tham chiếu để minh họa kiến trúc.

- Sơ đồ kiến trúc tổng quan: Hệ thống bao gồm các module chính: Module thu thập dữ liệu (camera), Module xử lý ALPR (YOLOv8 + EasyOCR), Module quản lý (quản lý xe vào/ra), và Module lưu trữ (MySQL). Các module kết nối qua giao thức MQTT để truyền dữ liệu thời gian thực.
- Sơ đồ luồng dữ liệu (Data Flow Diagram): Dữ liệu từ camera chảy vào ALPR pipeline, sau đó được xử lý và lưu vào cơ sở dữ liệu. Luồng ra bao gồm báo cáo và thông báo. (Sơ đồ có thể được vẽ với các thực thể: Nguồn dữ liệu → Xử lý → Lưu trữ → Đầu ra người dùng).
- Sơ đồ thực thể – quan hệ (ERD): Bao gồm các thực thể chính như Vehicle (biển số, loại xe), DetectionLog (thời gian, kết quả ALPR), Image (ảnh lưu trữ), và ParkingSlot (vị trí đỗ). Các quan hệ: Vehicle 1:N DetectionLog, DetectionLog 1:1 Image.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

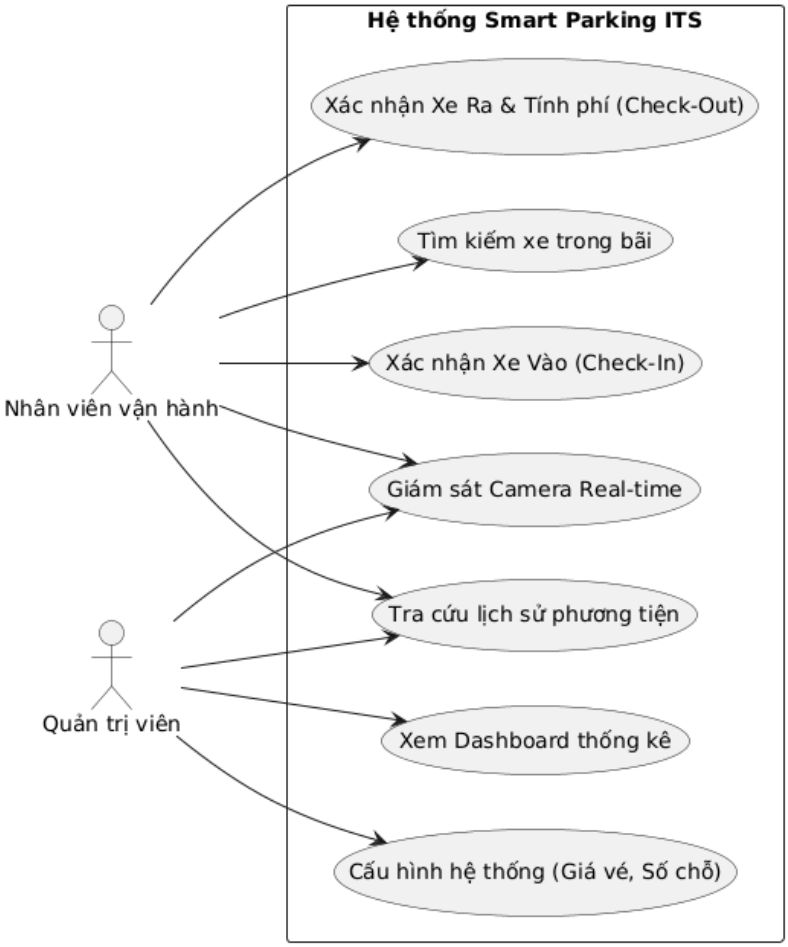
3.1. Tổng quan kiến trúc hệ thống

Hệ thống được xây dựng dựa trên kiến trúc Microservices đơn giản hóa, đóng gói trong môi trường Docker, đảm bảo tính độc lập giữa các thành phần và dễ dàng triển khai trên nhiều môi trường khác nhau. Hệ thống bao gồm 3 phân hệ chính:

1. Frontend (Giao diện người dùng): Phát triển bằng thư viện ReactJS kết hợp Tailwind CSS, cung cấp giao diện Dashboard hiện đại, tương tác thời gian thực với camera và dữ liệu.
2. Backend (Xử lý nghiệp vụ & AI): Phát triển bằng FastAPI (Python), đảm nhiệm vai trò tiếp nhận yêu cầu từ Frontend, xử lý hình ảnh qua mô hình YOLOv8/EasyOCR và giao tiếp với cơ sở dữ liệu.
3. Database (Lưu trữ dữ liệu): Sử dụng hệ quản trị MySQL 8.0 để lưu trữ thông tin xe, lịch sử ra vào và cấu hình hệ thống.

3.2. Sơ đồ Use Case (Biểu đồ ca sử dụng)

Sơ đồ dưới đây mô tả các chức năng chính mà người vận hành (Operator) và quản trị viên (Admin) có thể thực hiện trên hệ thống.



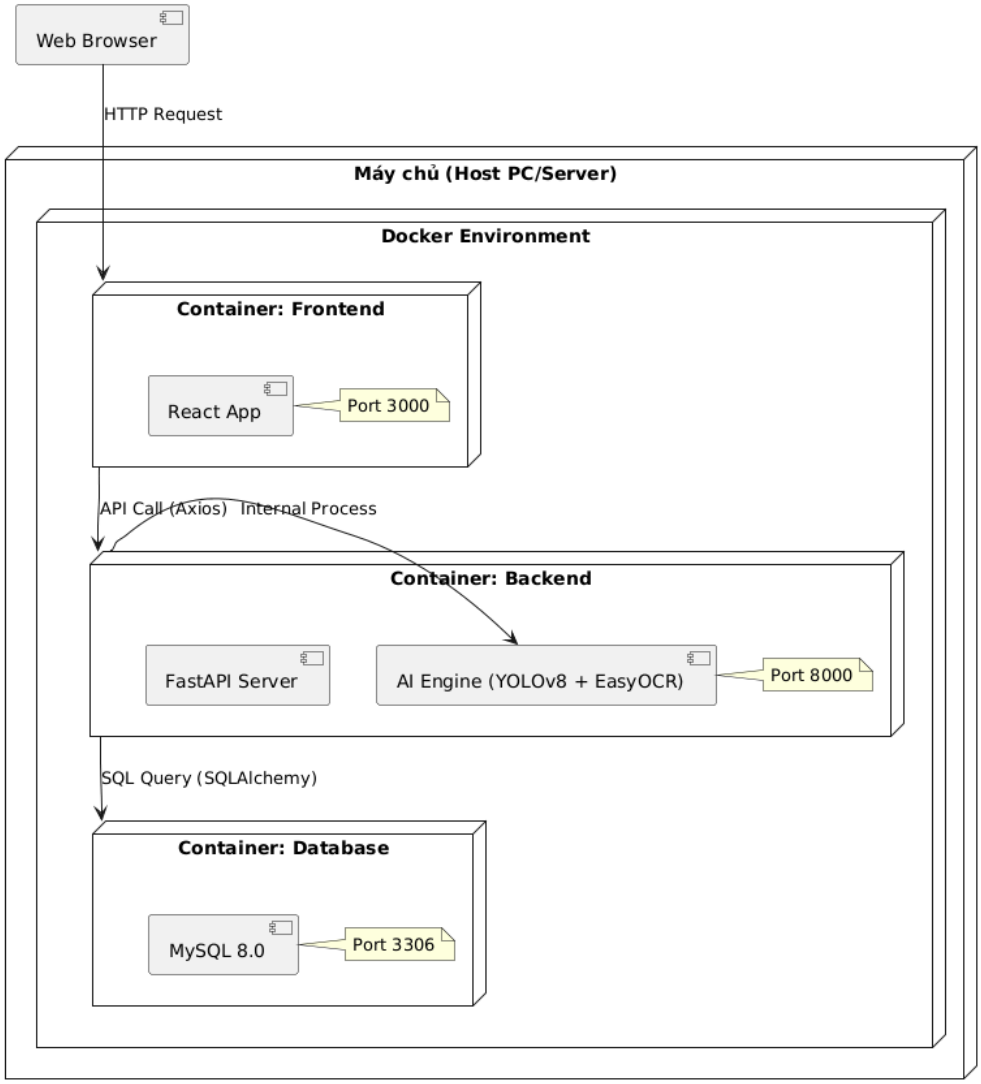
Hình 3.1: Use Case Diagram

Đặc tả các Use Case chính:

- Giám sát Camera: Hiển thị luồng video trực tiếp từ webcam, vẽ khung nhận diện biển số thời gian thực.
- Xác nhận Xe Vào: Khi AI phát hiện biển số mới, hệ thống hiển thị thông tin để nhân viên xác nhận lưu vào CSDL.
- Xác nhận Xe Ra: Khi quét lại biển số đã có trong bãi, hệ thống tự động tính toán thời gian đỗ và phí dịch vụ, sau đó cập nhật trạng thái xe.

3.3. Thiết kế kiến trúc và triển khai (Deployment Diagram)

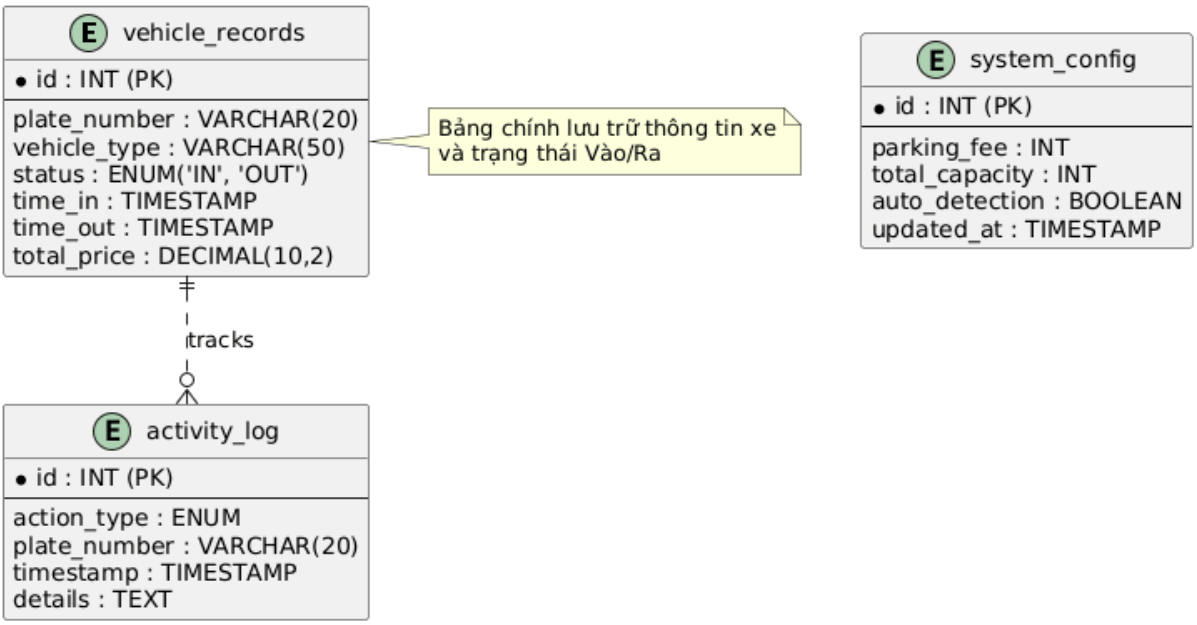
Hệ thống sử dụng Docker Compose để điều phối hoạt động của 3 container riêng biệt, giao tiếp qua mạng nội bộ Docker (Bridge Network).



Hình 3.2: Deployment Diagram

3.4. Thiết kế cơ sở dữ liệu (Database Design)

Cơ sở dữ liệu được thiết kế tối ưu cho việc truy vấn lịch sử và tính toán doanh thu.

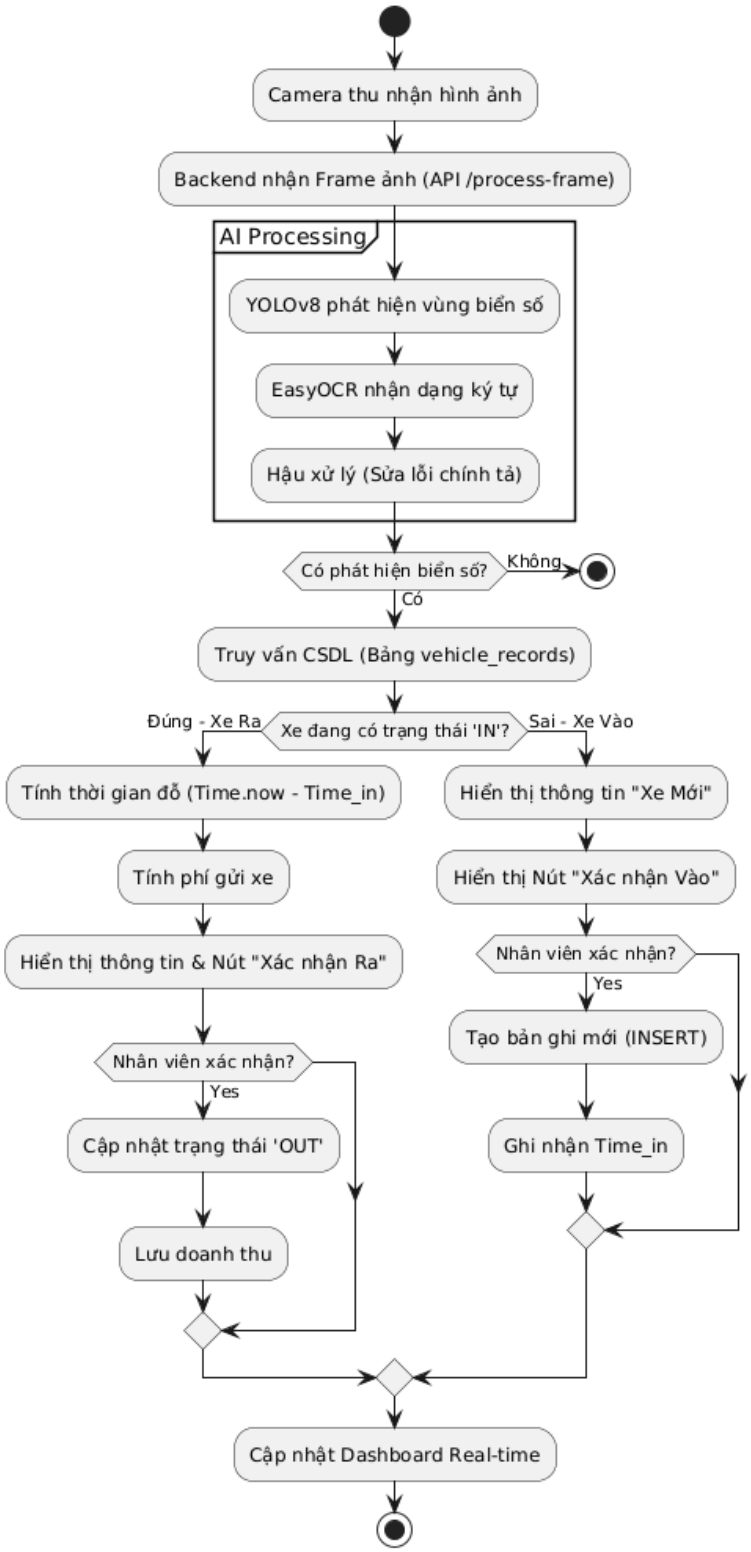


Hình 3.3: Cấu trúc database design

3.5. Thiết kế luồng xử lý (Activity & Sequence Diagrams)

3.5.1. Quy trình xử lý xe vào/ra tự động

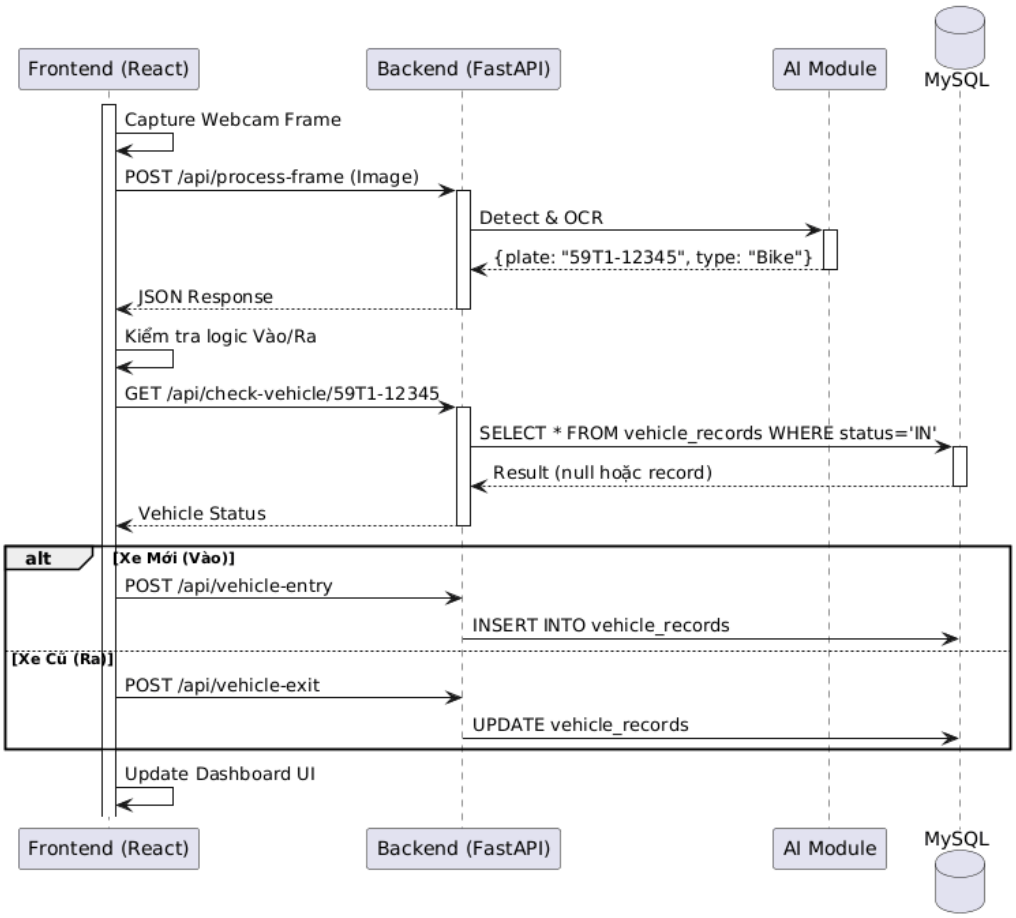
Đây là quy trình nghiệp vụ cốt lõi, nơi hệ thống tự động quyết định xem một biển số vừa quét là xe đang vào hay xe đang ra dựa trên dữ liệu lịch sử.



Hình 3.4: Sơ đồ quy trình xử lý xe vào/ra tự động

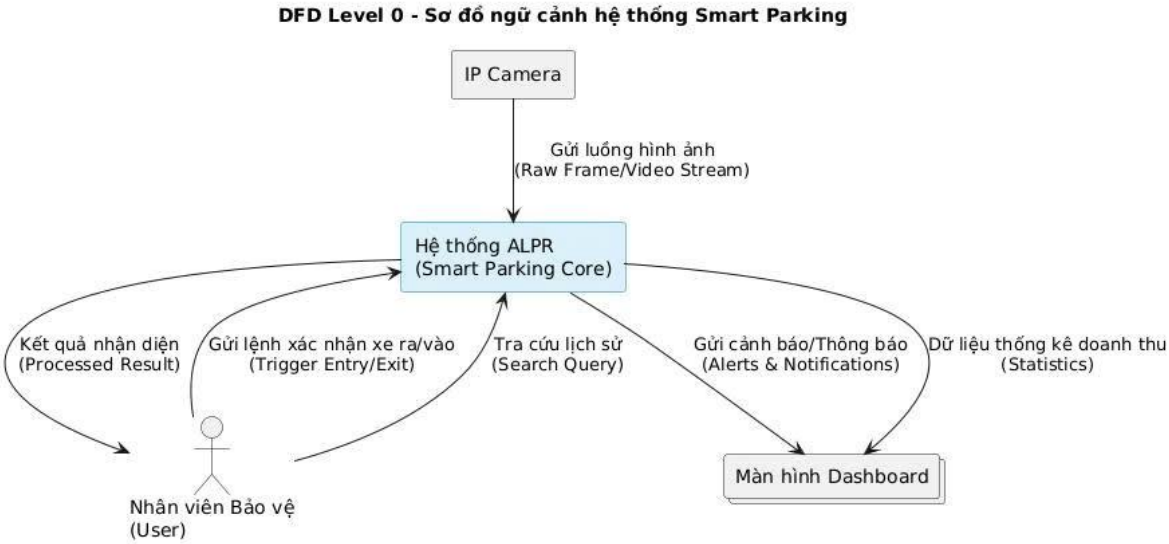
3.5.2. Biểu đồ tuần tự (Sequence Diagram) – API Call

Mô tả chi tiết cách Frontend tương tác với Backend khi xử lý một khung hình.

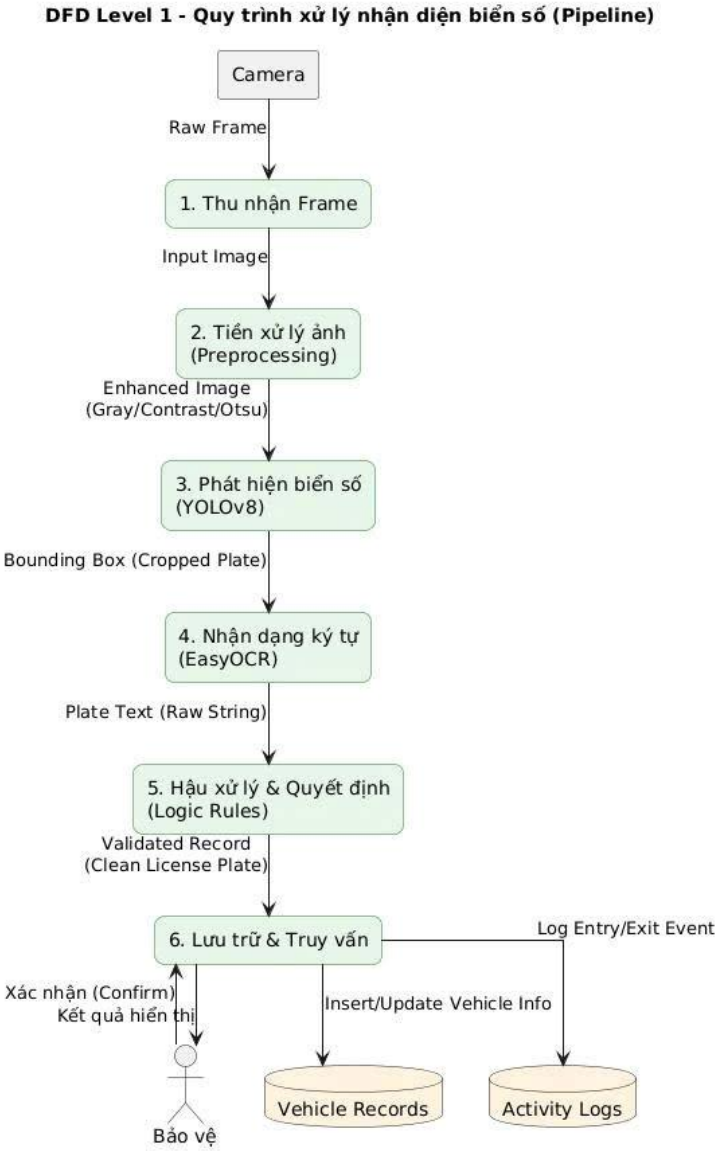


Hình 3.5: Sơ đồ tuần tự (Sequence Diagram)

3.6. Thiết kế luồng dữ liệu chi tiết (Data Flow Diagram - DFD)



Hình 3.6: Sơ đồ DFD Level 0



Hình 3.7: Sơ đồ DFD Level 1

3.7. Công nghệ sử dụng

Thành phần	Công nghệ / Thư viện	Phiên bản / Ghi chú
Frontend	ReactJS	v18.0
	Tailwind CSS	Styling Framework
	Axios	HTTP Client
Backend	Python	v3.10
	FastAPI	Web Framework hiệu năng cao
	SQLAlchemy	ORM cho truy vấn CSDL

AI Core	Ultralytics YOLOv8	Object Detection
	EasyOCR	Optical Character Recognition
	OpenCV	Xử lý ảnh (Image Processing)
Database	MySQL	v8.0
DevOps	Docker & Docker Compose	Đóng gói và triển khai

Bảng 3.1: Bảng tóm tắt các công nghệ và thư viện chính được sử dụng trong dự án

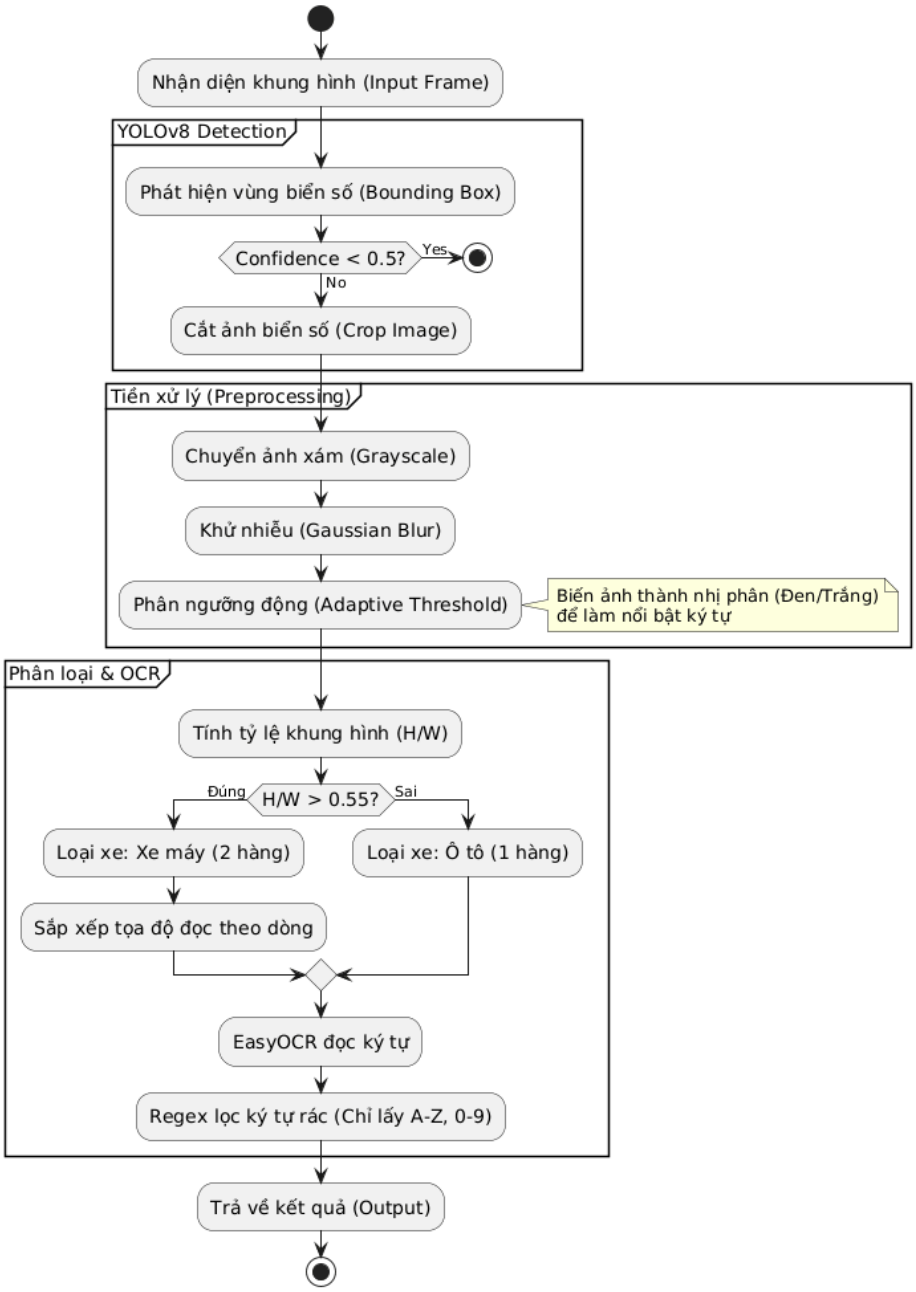
3.8. Đặc tả các giao thức kết nối (API Endpoints)

STT	Phương thức	Endpoint (Đường dẫn)	Chức năng	Input (Tham số)	Output (Trả về)
1	POST	/api/process-frame	Gửi ảnh từ camera để AI xử lý	File ảnh (Multipart)	JSON (Biển số, Loại xe, Tọa độ)
2	GET	/api/dashboard	Lấy số liệu thống kê cho Dashboard	Không có	JSON (Số xe vào, ra, doanh thu)
3	POST	/api/vehicle-entry	Xác nhận cho xe vào bãi	JSON (Biển số, Loại xe)	Message: "Success"
4	POST	/api/vehicle-exit	Xác nhận cho xe ra & Lưu doanh thu	JSON (Biển số)	Message: "Success"
5	GET	/api/history	Lấy danh sách xe đang trong bãi	Không có	JSON (List xe)

Bảng 3.2: Đặc tả các giao thức kết nối

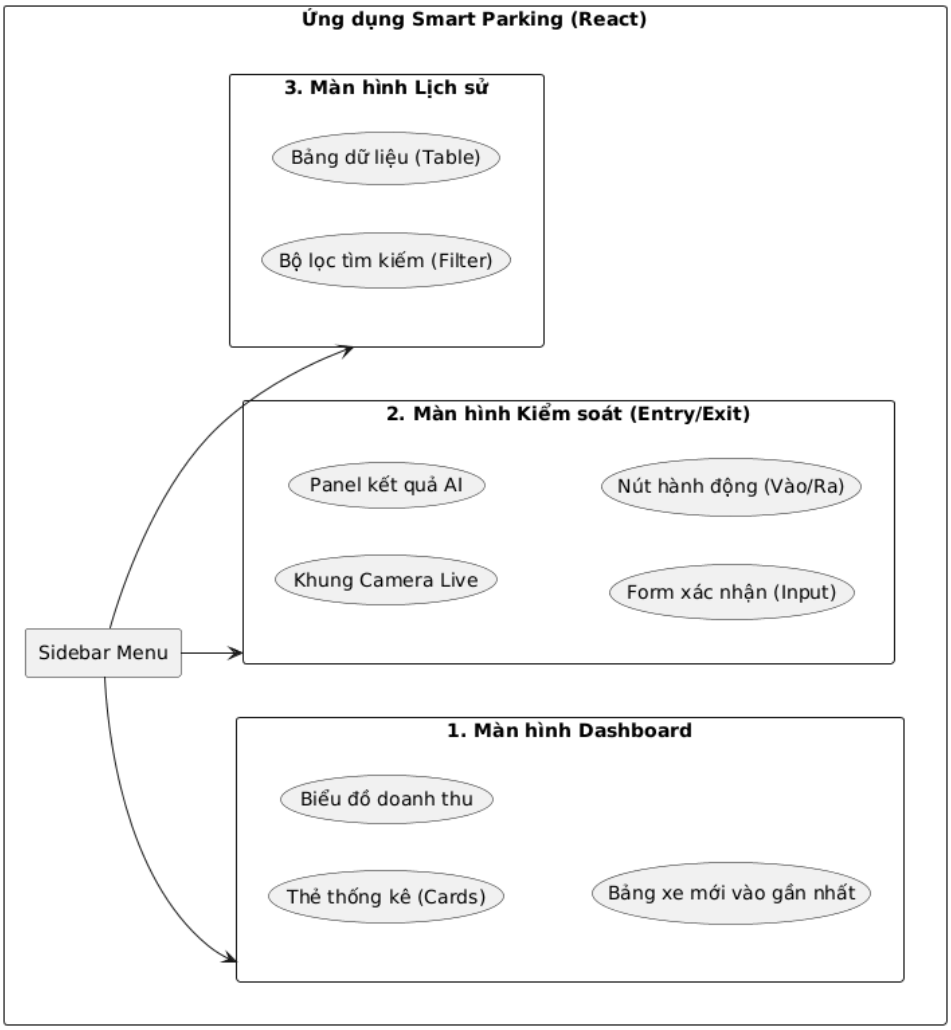
3.9. Chi tiết thuật toán tiền xử lý ảnh

Sơ đồ dưới đây chi tiết hóa các bước tiền xử lý để tăng độ chính xác cho EasyOCR.



Hình 3.8: Thuật toán tiền xử lý ảnh

3.10. Cấu trúc chức năng Frontend



Hình 3.9: Cấu trúc Frontend

CHƯƠNG 4: HIỆN THỰC VÀ TRIỂN KHAI

4.1. Giới thiệu và quá trình hiện thực hóa

Mục này đóng vai trò dẫn dắt người đọc từ lý thuyết sang thực tiễn. Nội dung chi tiết bao gồm:

- Mô tả tổng quan: Quá trình hiện thực hóa là giai đoạn chuyển đổi các sơ đồ thiết kế hệ thống (Chương 3) thành một sản phẩm phần mềm hoạt động được. Cụ thể, hệ thống sẽ được xây dựng dựa trên kiến trúc Microservices nhẹ, nơi các module AI (YOLOv8 & EasyOCR) đóng vai trò hạt nhân xử lý, kết hợp với giao thức MQTT để đảm bảo tính thời gian thực trong việc nhận tín hiệu từ camera bãi xe.
- Môi trường phát triển: Hệ thống được phát triển và thử nghiệm trên nền tảng Kaggle tận dụng sức mạnh của 2x GPU T4. Việc sử dụng GPU song song giúp tăng tốc đáng kể quá trình suy luận (inference) của mô hình nhận diện, đảm bảo xe không phải dừng quá lâu tại cổng.
- Mục tiêu của chương: Trình bày chi tiết cách cài đặt và cấu hình môi trường.
  + Giải thích các đoạn mã nguồn then chốt xử lý logic nhận diện và quản lý dữ liệu.
  + Chứng minh tính hiệu quả của hệ thống thông qua kết quả chạy thực tế (Demo).

4.2. Công cụ sử dụng

4.2.1. Ngôn ngữ và thư viện lập trình

- Python 3.8+: Là ngôn ngữ chủ đạo nhờ hệ sinh thái thư viện AI phong phú và khả năng xử lý bất đồng bộ tốt.
- OpenCV (Computer Vision): Sử dụng để tiền xử lý ảnh (cắt vùng biển số, khử nhiễu, cân bằng sáng) trước khi đưa vào module nhận dạng.
- Ultralytics YOLOv8: Lựa chọn mô hình yolov8n.pt (Nano) để tối ưu giữa độ chính xác và tốc độ phát hiện biển số trong khung hình.
- EasyOCR: Thay thế cho các giải pháp truyền thống (như Tesseract) vì EasyOCR dựa trên nền tảng Deep Learning (PyTorch), có khả năng nhận diện tốt các ký tự bị biến dạng nhẹ hoặc chụp dưới góc nghiêng.
- MySQL Connector: Đảm bảo kết nối ổn định và an toàn giữa mã nguồn Python và cơ sở dữ liệu MySQL, hỗ trợ lưu trữ hàng nghìn bản ghi xe ra vào.

4.2.2. Giao tiếp dữ liệu và Môi trường

- Giao thức MQTT: Sử dụng Broker (ví dụ: Mosquitto) để truyền gói tin chứa thông tin biển số từ Camera về Server. Ưu điểm là băng thông thấp và độ trễ cực nhỏ.
- Hardware (Phần cứng):
  + Huấn luyện: Tận dụng GPU T4 x2 (Kaggle) để xử lý song song các luồng video thử nghiệm.
  + Triển khai: Được tối ưu hóa để có thể chạy trên CPU của các máy tính quản lý tại bãi xe thông qua việc chuyển đổi model sang định dạng ONNX hoặc TensorRT.

- Công cụ hỗ trợ:
  + Roboflow: Quản lý tập dữ liệu ảnh biển số, thực hiện các kỹ thuật tăng cường dữ liệu (Augmentation) như xoay ảnh, thay đổi độ sáng để mô hình AI thích nghi tốt hơn với thực tế.

4.2.3. Bảng tóm tắt công cụ sử dụng

Nhóm công cụ	Tên công cụ / Thư viện	Phiên bản (Gợi ý)	Vai trò trong hệ thống
Ngôn ngữ	Python	3.9.x	Ngôn ngữ lập trình chính
Phát hiện (Detect)	YOLOv8	Ultralytics 8.0+	Phát hiện vị trí biển số xe
Nhận diện (OCR)	EasyOCR	1.7.x	Trích xuất ký tự từ vùng biển số
Xử lý ảnh	OpenCV	4.x	Tiền xử lý, hiển thị luồng video
Cơ sở dữ liệu	MySQL	8.0	Lưu trữ thông tin xe và log ra vào
Truyền tin	MQTT (Paho)	1.6.x	Giao tiếp thời gian thực
Phần cứng	NVIDIA GPU T4	16GB VRAM	Tăng tốc độ suy luận mô hình AI
Quản lý mã nguồn	Git / GitHub	-	Kiểm soát phiên bản và cộng tác

Bảng 4.1: Danh mục công nghệ và công cụ triển khai hệ thống

4.3. Mô tả các Module chính

4.3.1. Module Tiền xử lý & OCR

```
def preprocess_for_ocr(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.resize(gray, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_CUBIC)

    # Sử dụng Threshold Otsu để tách biệt hẳn nét chữ và nền
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
    gray = clahe.apply(gray)

    _, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return thresh
```

Hình 4.1: Hàm tiền xử lý ảnh sử dụng thuật toán CLAHE và Otsu Thresholding để tách nền.

- Tiền xử lý: Sử dụng OpenCV để thực hiện các bước:
  + Grayscale & Blur: Chuyển ảnh biển số về mức xám và áp dụng cv2.GaussianBlur để giảm nhiễu bề mặt biển số (bụi bẩn, vết xước).

+ Thresholding: Sử dụng cv2.threshold với phương pháp Otsu để tách biệt rõ rệt ký tự đen trên nền trắng, giúp tăng độ tương phản cho bước OCR.

- OCR (EasyOCR): Thay thế EasyOCR bằng thư viện EasyOCR.
  + Lý thuyết thực thi: Khởi tạo easyocr.Reader(['en']). Sử dụng mô hình Deep Learning dựa trên kiến trúc ResNet và LSTM để nhận diện chuỗi ký tự.
  + Tối ưu hóa: Thiết lập tham số allowlist='0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ' để ép mô hình chỉ nhận diện ký tự chữ và số, loại bỏ các ký hiệu nhiễu.

4.3.2. Module Hậu xử lý

```python
def final_correction(text):
    text = re.sub(r'[^A-Z0-9]', '', text.upper())
    if len(text) < 5:
        return text

    chars = list(text)

    # === 1. Hai ký tự đầu: PHẢI LÀ SỐ ===
    to_num = {
        'I': '1', 'J': '1', 'L': '1',
        'O': '0', 'D': '0', 'Q': '0',
        'B': '8', 'S': '5'
    }

    for i in [0, 1]:
        if i < len(chars) and chars[i] in to_num:
            chars[i] = to_num[chars[i]]

    # === 2. Ký tự thứ 3: PHẢI LÀ CHỮ ===
    to_char = {
        '0': 'D',
        '1': 'I',
        '2': 'Z',
        '4': 'A',
        '5': 'S',
        '8': 'B'
    }

    if len(chars) > 2 and chars[2].isdigit() and chars[2] in to_char:
        chars[2] = to_char[chars[2]]

    # === 3. Các ký tự sau: PHẢI LÀ SỐ ===
    for i in range(3, len(chars)):
        if chars[i] in to_num:
            chars[i] = to_num[chars[i]]

    return "".join(chars)
```

Hình 4.2: Thuật toán hậu xử lý (Post-processing) sử dụng Regex và Dictionary Mapping để sửa lỗi nhận diện.

- Chuẩn hóa dữ liệu: Sử dụng Regular Expression (Regex) để định dạng lại chuỗi văn bản từ EasyOCR (ví dụ: tự động thêm dấu gạch ngang giữa sê-ri và số thứ tự: 29A12345 -> 29A-12345).

- Lọc nhiễu kết quả: Áp dụng ngưỡng tin cậy confidence_score. Chỉ những kết quả có độ tin cậy > 0.7 mới được gửi tới Module Quản lý.

4.3.3. Module quản lý xe vào/ra

```python
@app.get("/api/check-vehicle/{plate_number}")
async def check_vehicle(plate_number: str):
    """Check if a vehicle is currently in the parking lot"""
    try:
        with engine.connect() as conn:
            result = conn.execute(
                text("""
                    SELECT id, plate_number, time_in, status
                    FROM vehicle_records
                    WHERE plate_number = :plate AND status = 'IN'
                    ORDER BY id DESC
                    LIMIT 1
                """),
                {"plate": plate_number}
            ).fetchone()

            if result:
                return {
                    "exists": True,
                    "id": result[0],
                    "plate_number": result[1],
                    "time_in": result[2].isoformat() if result[2] else None,
                    "status": result[3]
                }
            else:
                return {"exists": False}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Hình 4.3: Hàm kiểm tra xem biển số này đã có trong bãi chưa

- Logic điều khiển: Khi nhận được tín hiệu từ MQTT (từ camera), module sẽ gọi hàm nhận diện. Nếu biển số hợp lệ, module thực hiện truy vấn trạng thái xe (đã có trong bãi hay chưa) để quyết định mở rào chắn.
- Tích hợp phần cứng: Gửi lệnh qua giao thức MQTT để điều khiển mô phỏng rào chắn (barrier).

4.3.4. Module quản lý dữ liệu

Query 1 vehicle_records

```
1 • SELECT * FROM smart_parking_its.vehicle_records;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	plate_number	vehicle_type	time_in	time_out	status	total_price
28	29Y303658	Biển vuông	2026-01-24 12:31:24	2026-01-24 12:32:47	OUT	5000.00
29	99E122268	Biển vuông	2026-01-24 13:18:21	2026-01-24 13:21:39	OUT	5000.00
30	98K102897	Biển vuông	2026-01-24 15:43:39	NULL	IN	0.00
31	99E132268	Biển vuông	2026-01-24 16:17:17	NULL	IN	0.00
32	99E122268	Biển vuông	2026-01-24 16:17:17	2026-01-25 19:27:35	OUT	5000.00
33	30E77796	Biển vuông	2026-01-24 16:18:03	NULL	IN	0.00
34	9GE132268	Biển vuông	2026-01-24 16:43:08	NULL	IN	0.00
35	29Y303658	Biển vuông	2026-01-25 10:53:22	2026-01-25 11:33:01	OUT	5000.00
36	29Y303658	Biển vuông	2026-01-25 11:49:06	NULL	IN	0.00
37	89E118896	Biển vuông	2026-01-25 19:20:03	2026-01-25 20:07:57	OUT	5000.00
38	59V179379	Biển vuông	2026-01-25 19:29:18	NULL	IN	0.00
39	51H91991	Biển dài	2026-01-25 19:31:30	NULL	IN	0.00
40	89E8896	Biển vuông	2026-01-25 20:13:16	NULL	IN	0.00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Hình 4.4: Dữ liệu phương tiện được lưu trữ thành công trong bảng MySQL.

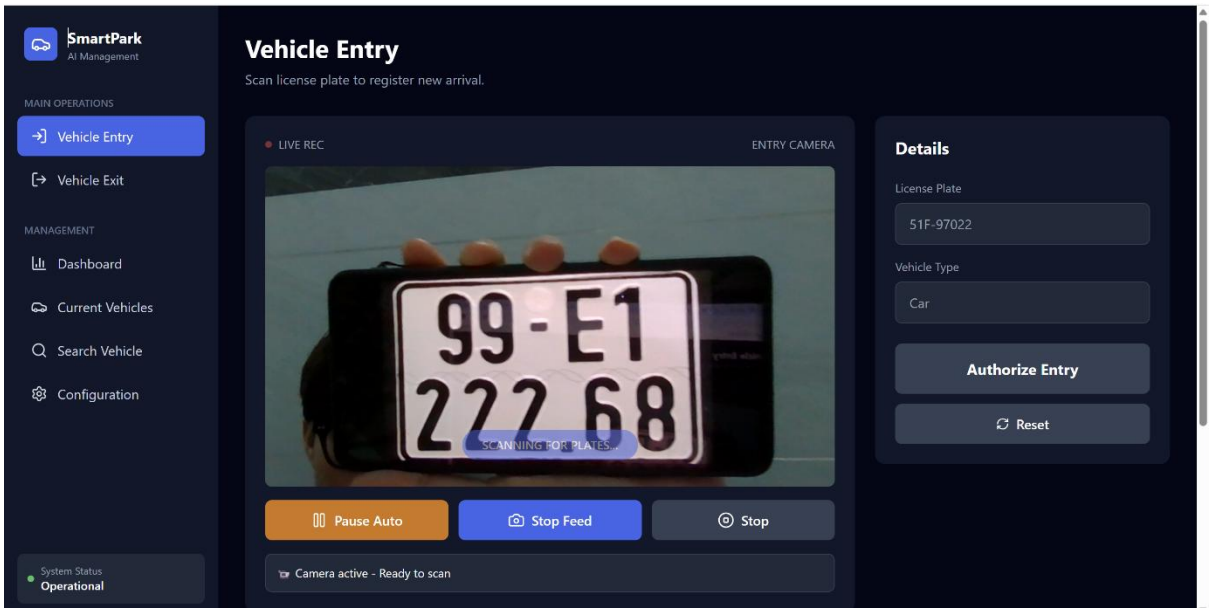
```python
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+mysqlconnector://root:long12345@db:3306/smart_parking_its"
)

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
```

Hình 4.5: kết nối DB.

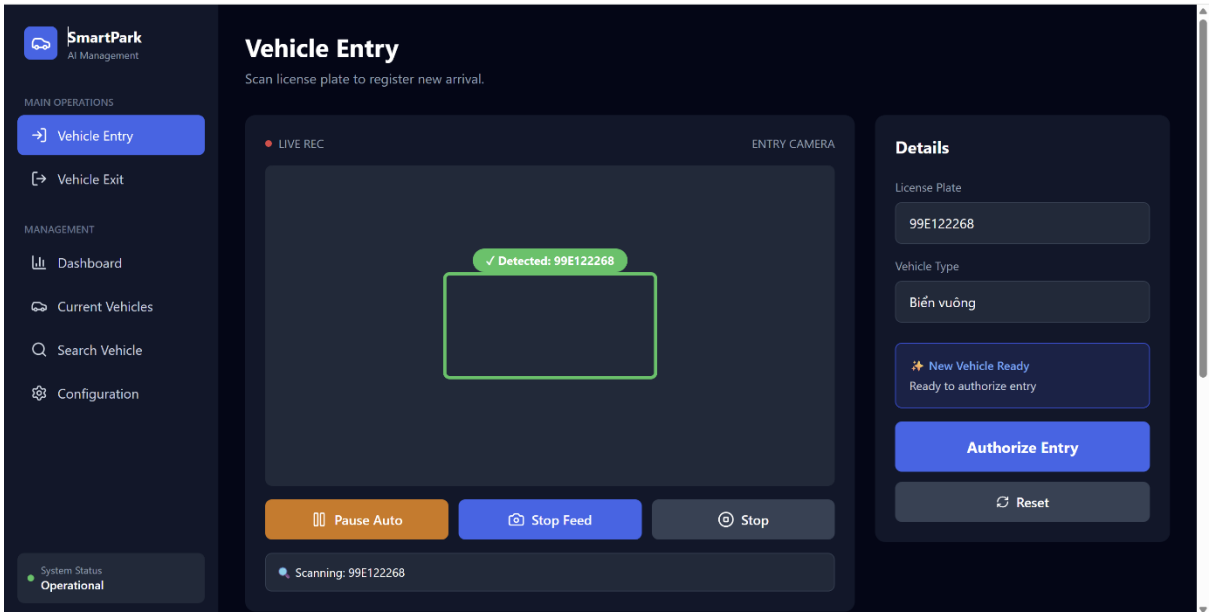
- Kết nối Database: Sử dụng mysql-connector-python để thực hiện các câu lệnh SQL (INSERT/UPDATE).
- Cấu trúc lưu trữ: Lưu trữ thông tin xe, thời gian vào/ra và đường dẫn ảnh biển số để phục vụ việc đối soát sau này.

4.4. Screenshot/Result Images

- Giao diện khi có xe vào:

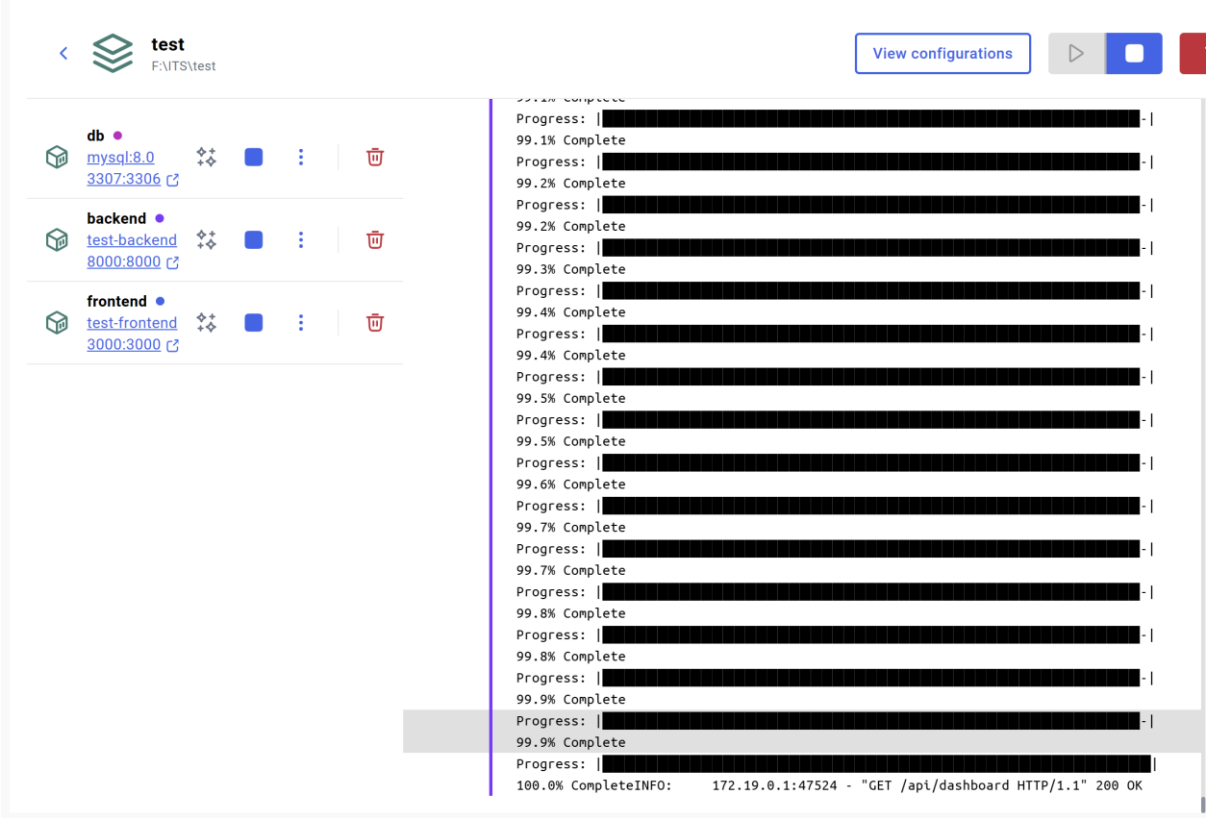


Hình 4.6: Giao diện dữ liều ảnh đầu vào.



Hình 4.7: Giao diện hiển thị thông tin xe vào và biển số đã nhận diện.

- Log Terminal của Docker:




```
smart_parking_backend   | INFO:     172.19.0.1:59564 - "GET /api/check-vehicle/99E122268 HTTP/1.1" 200 OK
smart_parking_backend   | INFO:     172.19.0.1:59570 - "GET /api/check-vehicle/99E122268 HTTP/1.1" 200 OK
smart_parking_backend   | INFO:     172.19.0.1:59564 - "GET /api/vehicles HTTP/1.1" 200 OK
smart_parking_backend   | INFO:     172.19.0.1:59570 - "GET /api/dashboard HTTP/1.1" 200 OK
smart_parking_backend   | INFO:     172.19.0.1:41628 - "GET /api/recent-history HTTP/1.1" 200 OK
```

Hình 4.8: Nhật ký hoạt động (Log) của các Container Backend và Database trong quá trình vận hành.

4.5. How design → Code mapping

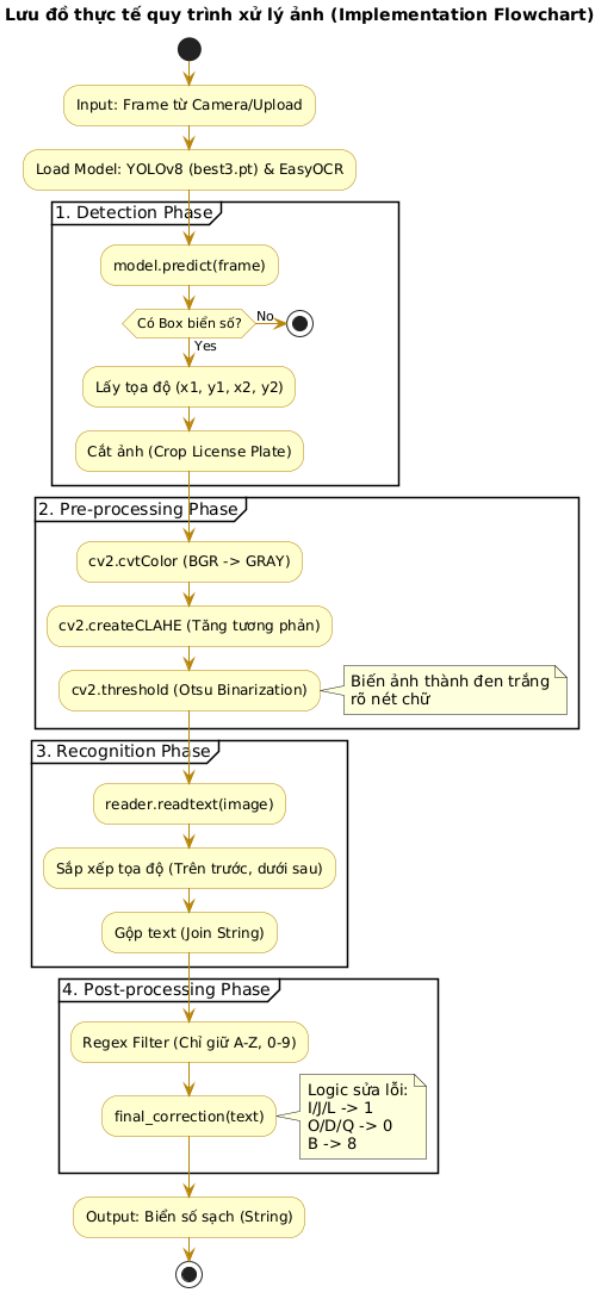
Thành phần thiết kế (Chương 3)	Thành phần hiện thực (Code thực tế)	Minh chứng (Hình ảnh/Module)
Use Case: Quản lý xe vào	Logic xử lý luồng xe và kiểm tra trạng thái	Hình 4.3 (Hàm kiểm tra biển số)
Thực thể dữ liệu (Vehicle)	Bảng MySQL lưu trữ thông tin phương tiện	Hình 4.4 (Bảng dữ liệu MySQL)
Module AI: Phát hiện biển số	Sử dụng thư viện YOLOv8 (Ultralytics)	Mục 4.3.1 (Module Tiền xử lý & OCR)
Module AI: Nhận diện ký tự	Thư viện **EasyOCR** (kết hợp Regex xử lý lỗi)	Hình 4.2 (Code xử lý OCR)
Giao thức truyền tin (MQTT)	Kết nối và nhận tín hiệu từ Camera	Mục 4.3.3 (Module Quản lý xe vào/ra)

Kết nối Cơ sở dữ liệu	Module `mysql-connector-python`	Hình 4.5 (Kết nối DB)
Giao diện Giám sát (UI)	Giao diện hiển thị dữ liệu xe thực tế	Hình 4.6 (Giao diện khi có xe vào)
Hậu xử lý dữ liệu	Thuật toán Regex định dạng biển số	Mục 4.3.2 (Module Hậu xử lý)

Bảng 4.2: Ánh xạ từ Thiết kế (Chương 3) đến Hiện thực (Chương 4)

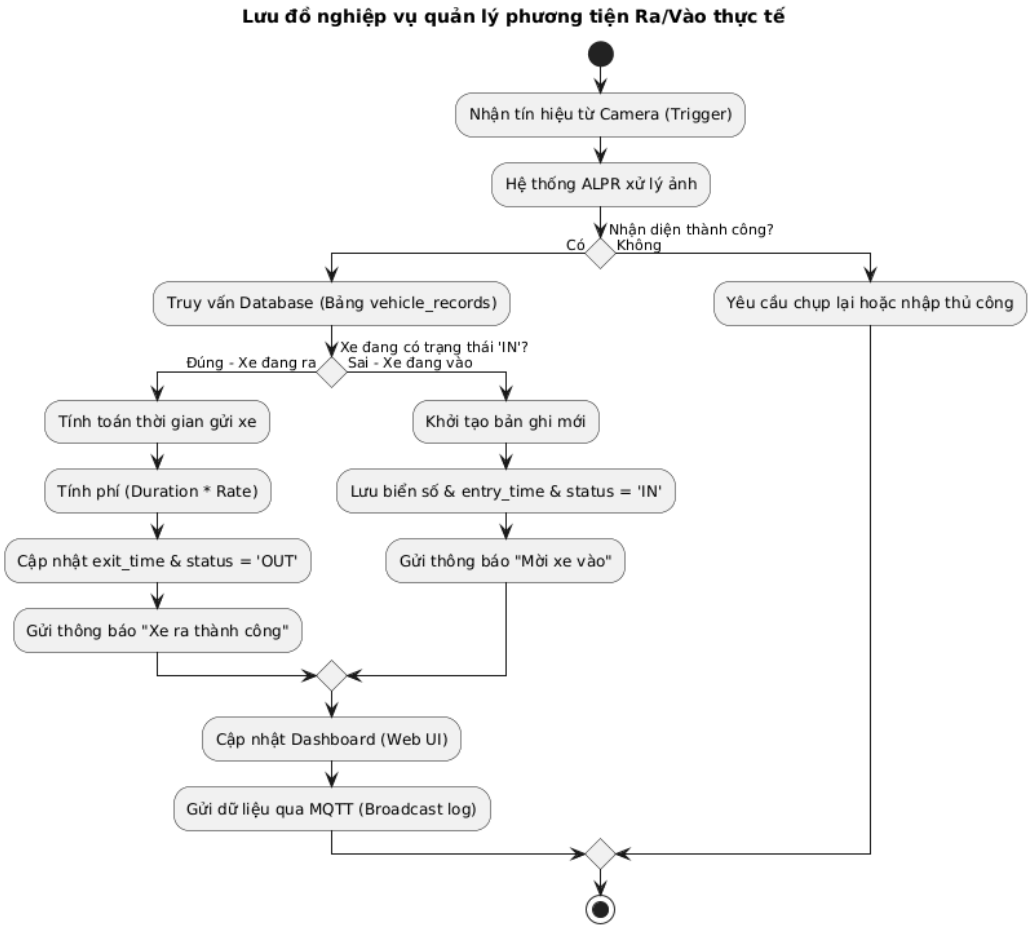
4.6. Flowchart thực tế

- Lưu đồ thuật toán thực tế trong processor.py:



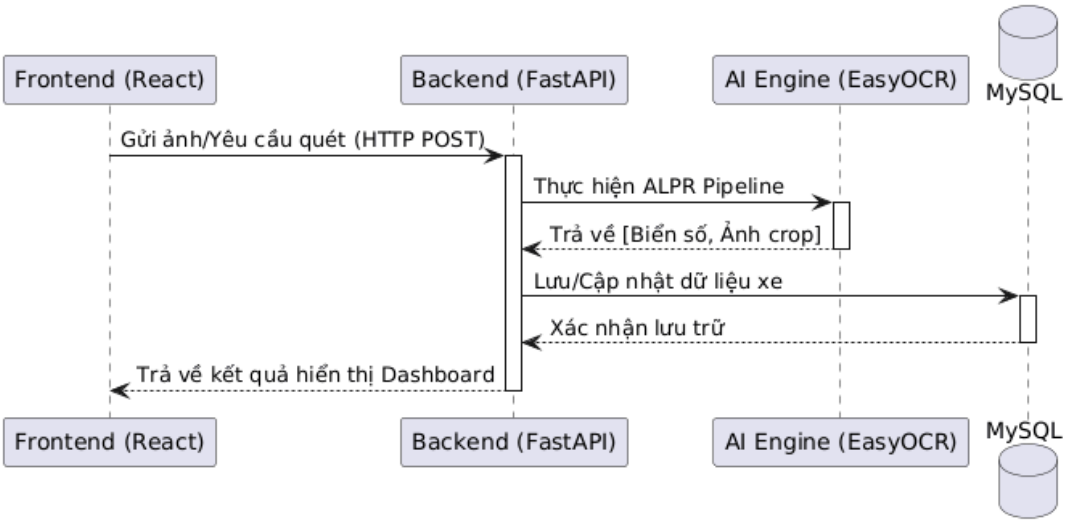
Hình 4.9: Lưu đồ thực tế quy trình xử lý ảnh

- Sơ đồ thể hiện quy trình quản lý bãi xe thực tế:



Hình 4.10: Flowchart Nghiệp vụ Xe Vào/Ra (Tích hợp DB).

- Sơ đồ mô tả cách các phần (Frontend - Backend - AI) phối hợp khi hệ thống đang chạy:



Hình 4.11: Sơ đồ hoạt động của hệ thống sau triển khai (Deployment Flow).

CHƯƠNG 5: KIỂM THỬ - ĐÁNH GIÁ

5.1. Giới thiệu về quá trình kiểm thử và đánh giá

Chương này trình bày quá trình kiểm thử và đánh giá toàn diện hệ thống nhận diện biển số xe tự động (Automatic License Plate Recognition – ALPR) được xây dựng trong đề tài. Nội dung của chương được tổng hợp, chọn lọc và tái cấu trúc từ các kết quả thực nghiệm đã trình bày chi tiết ở Chương 4, nhằm đáp ứng đúng yêu cầu kiểm thử hệ thống theo nhiều bối cảnh và đánh giá hiệu năng thực tế.

Mục tiêu chính của quá trình kiểm thử bao gồm:

- Đánh giá độ chính xác nhận dạng của từng thành phần (YOLOv8, EasyOCR) và toàn bộ hệ thống end-to-end.
- Đánh giá hiệu năng xử lý thông qua các chỉ số latency, throughput và FPS.
- Kiểm tra khả năng hoạt động của hệ thống trong các điều kiện thực tế khác nhau.
- Phân tích các trường hợp lỗi và hạn chế còn tồn tại.

Quá trình kiểm thử được thực hiện trên tập dữ liệu kiểm thử gồm 200 ảnh biển số xe Việt Nam, tách biệt hoàn toàn với dữ liệu huấn luyện, đảm bảo tính khách quan trong đánh giá.

5.2. Các trường hợp kiểm thử (test cases)

5.2.1. Tổng quan tập dữ liệu kiểm thử

Tập dữ liệu sử dụng cho quá trình kiểm thử bao gồm 200 ảnh biển số xe Việt Nam. Các ảnh trong tập test có các đặc điểm sau:

- Bao gồm biển số 1 dòng và 2 dòng
- Đa dạng về góc chụp, khoảng cách và điều kiện môi trường
- Có nhãn ground truth đầy đủ để so sánh với kết quả nhận dạng của hệ thống

Việc sử dụng tập test riêng biệt giúp đánh giá chính xác khả năng tổng quát hóa của mô hình khi áp dụng cho dữ liệu chưa từng xuất hiện trong quá trình huấn luyện.

5.2.2. Test cases theo điều kiện ánh sáng

5.2.2.1. Điều kiện ánh sáng tốt (ban ngày)

Các test cases trong nhóm này bao gồm các ảnh được chụp trong điều kiện:

- Ánh sáng tự nhiên đầy đủ.
- Biển số rõ nét, ít nhiễu.
- Độ tương phản cao giữa biển số và nền.

Kết quả kiểm thử cho thấy:

- Mô hình YOLOv8 phát hiện chính xác vị trí biển số với confidence cao.
- Vùng ảnh biển số sau khi cắt đảm bảo đầy đủ thông tin cho bước OCR.
- EasyOCR nhận dạng ký tự chính xác, tỷ lệ lỗi thấp.

Hình 5.1: Test case trong điều kiện ánh sáng tốt

5.2.2.2. Điều kiện ánh sáng kém (ban đêm, ngược sáng)

Nhóm test cases này bao gồm các ảnh:

- Chụp ban đêm dưới ánh đèn đường hoặc đèn xe.
- Có hiện tượng ngược sáng (backlight) hoặc vùng sáng – tối không đồng đều.

Trong các trường hợp này:

- YOLOv8 vẫn duy trì khả năng phát hiện biển số ổn định.
- Tuy nhiên, chất lượng ảnh đầu vào cho OCR giảm, dẫn đến khả năng nhầm lẫn một số ký tự có hình dạng tương đồng.

Hình 5.2: Test case trong điều kiện ánh sáng kém

5.2.3. Test cases theo mật độ phương tiện

5.2.3.1. Mật độ phương tiện thấp (xe đơn lẻ)

Trong các ảnh chỉ chứa một phương tiện:

- Biển số không bị che khuất.
- Bounding box được xác định chính xác.

Hệ thống đạt độ chính xác cao và ổn định trong toàn bộ pipeline, từ phát hiện đến nhận dạng ký tự.

Hình 5.3: Mật độ phương tiện thấp

5.2.3.2. Mật độ phương tiện cao (nhiều xe trong cùng khung hình)

Các test cases này mô phỏng bối cảnh:

- Bãi xe đông đúc hoặc khu vực ùn tắc.
- Nhiều phương tiện xuất hiện đồng thời trong ảnh.

Kết quả kiểm thử cho thấy:

- YOLOv8 vẫn phát hiện đúng biển số cần thiết.
- Trong một số trường hợp, biển số bị che khuất một phần bởi xe khác, làm giảm độ chính xác của OCR.

Hình 5.4: Mật độ phương tiện cao

5.2.4. Test cases occlusion và điều kiện bất lợi

Nhóm test cases này được tổng hợp trực tiếp từ phân tích lỗi ở Chương 4, bao gồm:

- Biển số bị che khuất một phần (20–40%).
- Biển số bị bẩn hoặc trầy xước.
- Ảnh có motion blur do xe di chuyển nhanh.

Đây là các nguyên nhân chính gây ra các lỗi nhận dạng ký tự trong hệ thống, mặc dù bước phát hiện biển số vẫn hoạt động ổn định.

Hình 5.5: Test cases occlusion

5.3. Các chỉ số đánh giá (Metrics)

5.3.1. Metrics đánh giá mô hình phát hiện biển số (YOLOv8)

Tham số	Ý nghĩa
data	Đường dẫn đến file dataset.yaml (định nghĩa train/val/test + danh sách classes)
epochs=60	Số vòng huấn luyện của mô hình
imgsz=640	Kích thước ảnh đầu vào, tất cả ảnh được resize về 640 × 640
batch=110	Batch size = 110
device="cuda"	Sử dụng GPU để tăng tốc quá trình huấn luyện

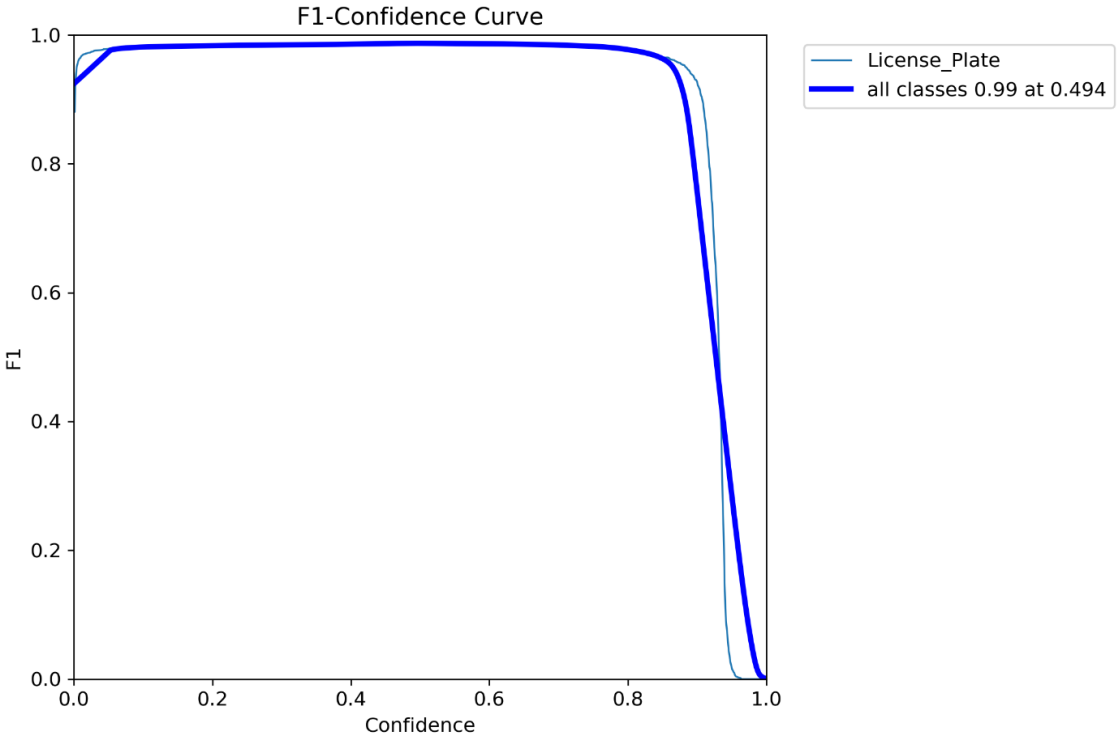
Bảng 5.1: Tham số training model

Chỉ số	Giá trị	Ý nghĩa thực tiễn
mAP@0.5	0.995	Mô hình phát hiện biển số với độ chính xác rất cao ở ngưỡng IoU 0.5
mAP@0.5–0.95	0.995	Khả năng định vị khung bao chính xác và ổn định trên nhiều mức IoU
Precision	1.00	Không phát sinh phát hiện sai, đảm bảo độ tin cậy cao trong vận hành
Recall	1.00	Không bỏ sót biển số trong dữ liệu đánh giá
F1-Score	≈ 0.99	Thể hiện sự cân bằng tối ưu giữa Precision và Recall

Bảng 5.2: chỉ số YOLOv8

Các đường cong đánh giá hiệu suất YOLOv8:

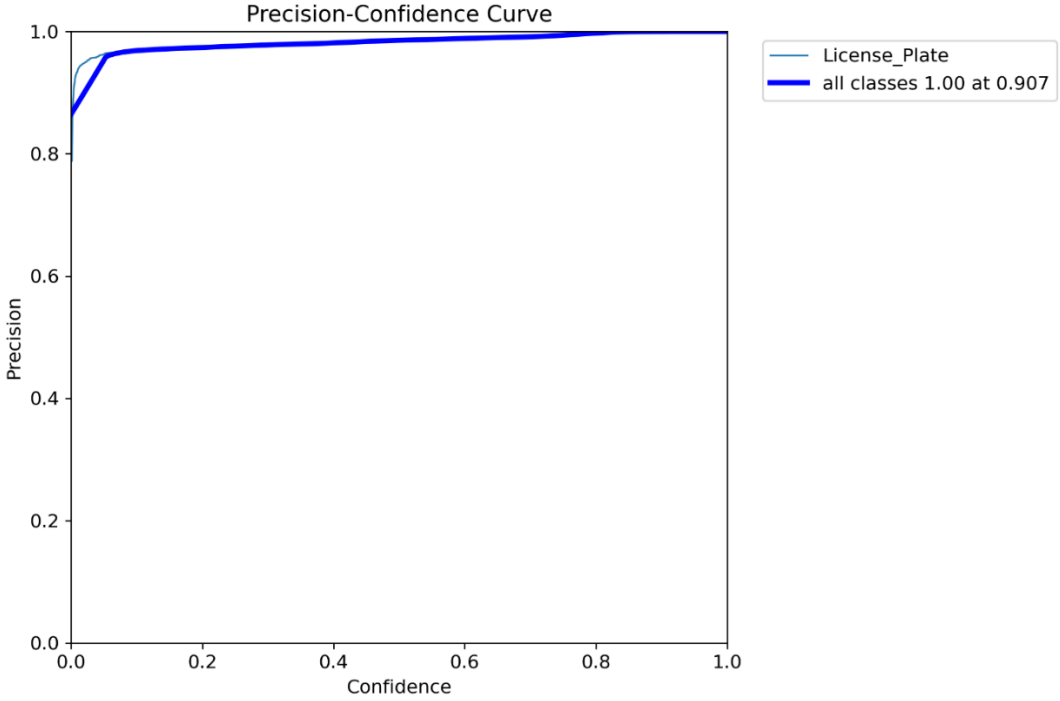
5.3.1.1. Đường cong F1-Confidence: Chứng thực sự ổn định



Hình 5.6: F1-Confidence Curve (YOLO)

- Phân tích: Đường cong F1–Confidence cho thấy mô hình duy trì giá trị F1 rất cao (gần 1.0) trên một dải ngưỡng tin cậy rộng, từ 0.0 đến khoảng 0.9. Giá trị F1 tối ưu đạt 0.99 tại ngưỡng confidence ≈ 0.494, phản ánh sự cân bằng tốt giữa Precision và Recall.
- Ý nghĩa thực tiễn: Kết quả này chứng tỏ mô hình có độ ổn định cao và linh hoạt khi triển khai thực tế. Ngưỡng confidence có thể được điều chỉnh để ưu tiên Recall hoặc Precision mà không làm suy giảm đáng kể hiệu suất tổng thể của hệ thống.

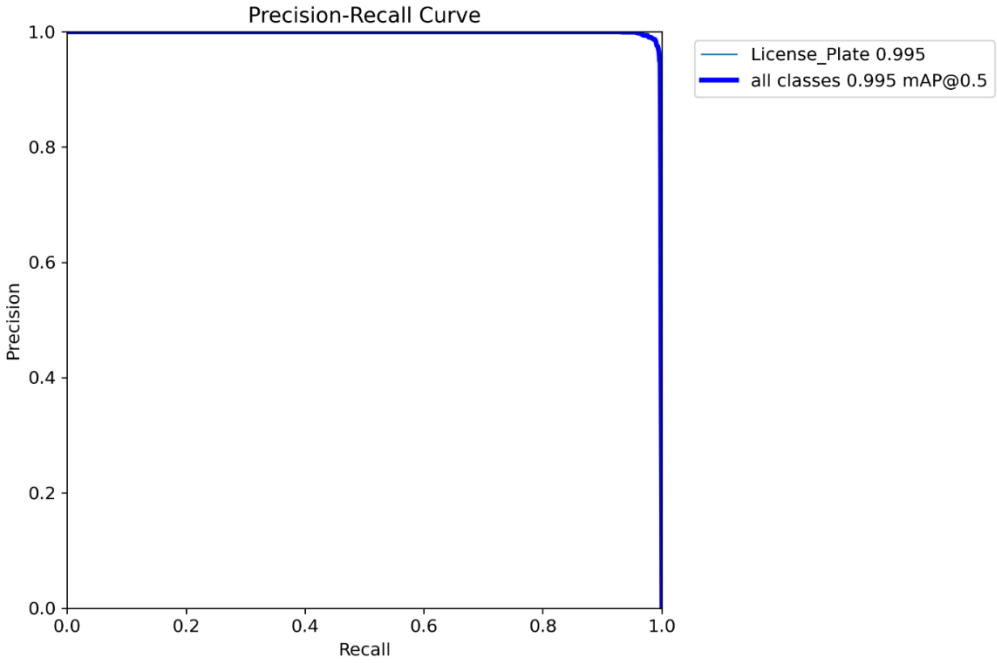
5.3.1.2. Đường cong Precision-Confidence: Đảm bảo Độ tin cậy Cực cao



Hình 5.7: Precision-Confidence Curve (YOLO)

- Phân tích: Precision duy trì ở mức 1.00 hoàn hảo bắt đầu từ ngưỡng tin cậy 0.907 và luôn ở mức rất cao trên toàn bộ dải. Ngay cả ở các ngưỡng tin cậy thấp, độ chính xác vẫn vượt trội.
- Ý nghĩa thực tiễn: Đặc tính này làm cho mô hình trở thành lựa chọn lý tưởng cho các ứng dụng không khoan nhượng với sai sót (mission-critical). Mọi phát hiện do mô hình đưa ra, đặc biệt là ở các ngưỡng tin cậy trung bình và cao, đều có độ tin cậy cao nhất, giảm thiểu rủi ro hoạt động.

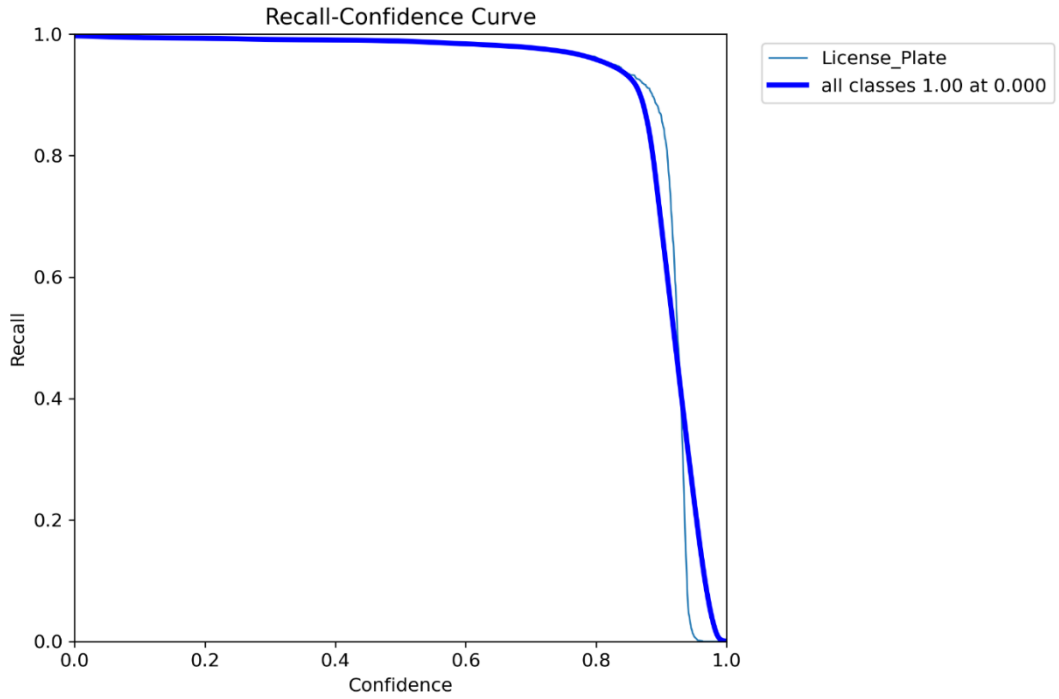
5.3.1.3. Đường cong Precision-Recall: Minh chứng cho Khả năng Phân biệt Vượt trội



Hình 5.8: Precision-Recall Curve (YOLO)

- Phân tích: Đường cong Precision–Recall cho thấy cả hai chỉ số đều duy trì ở mức rất cao trên hầu hết miền giá trị, với đường cong áp sát các giá trị biên tối đa. Dạng đường cong này phản ánh khả năng phân tách đối tượng rất tốt của mô hình, đồng thời phù hợp với giá trị mAP@0.5 đạt 0.995, cho thấy diện tích dưới đường cong gần đạt mức tối ưu.
- Ý nghĩa thực tiễn: Hình dạng này chứng tỏ mô hình có thể đạt được độ bao phủ (Recall) gần như hoàn hảo mà không phải hy sinh độ chính xác (Precision). Điều này cho thấy mô hình đã học được các đặc trưng có khả năng phân biệt cao, tránh được sự đánh đổi phổ biến nơi việc tăng Recall dẫn đến nhiều cảnh báo sai (false positive).

5.3.1.4. Đường cong Recall-Confidence: Đánh giá Độ nhạy Toàn diện



Hình 5.9: Recall-Confidence Curve (YOLO)

- Phân tích: Đường cong cho thấy Recall đạt giá trị tuyệt đối 1.0 ở các ngưỡng tin cậy thấp và duy trì ở mức rất cao (trên 0.95) cho đến khi ngưỡng tin cậy đạt khoảng 0.85.
- Ý nghĩa thực tiễn: Mô hình có độ nhạy rất cao, có khả năng phát hiện gần như toàn bộ các biển số xe có trong ảnh. Đặc tính này rất quan trọng cho các ứng dụng mà việc bỏ sót một đối tượng là không thể chấp nhận được, chẳng hạn như trong các hệ thống giám sát an ninh hoặc kiểm soát truy cập.

5.3.2. Metrics đánh giá mô hình OCR (EasyOCR)

Trong hệ thống nhận dạng biển số xe tự động (ALPR), mô hình OCR đóng vai trò quyết định trong việc trích xuất chính xác chuỗi ký tự từ vùng biển số đã được phát hiện. Do đó, việc lựa chọn các chỉ số đánh giá phù hợp là yếu tố quan trọng nhằm phản ánh đúng hiệu quả và khả năng ứng dụng thực tế của mô hình EasyOCR.

Trong đề tài này, mô hình EasyOCR được đánh giá dựa trên các chỉ số chuẩn trong bài toán OCR, tập trung vào độ chính xác ký tự và hiệu năng suy luận, phù hợp với mục tiêu triển khai trong hệ thống ITS.

5.3.2.1. Các chỉ số đánh giá chính

Character Error Rate (CER): là chỉ số quan trọng nhất được sử dụng để đánh giá hiệu năng của mô hình EasyOCR trong đề tài. CER được định nghĩa là tỷ lệ số ký tự bị nhận dạng sai so với tổng số ký tự trong dữ liệu ground truth.

Chỉ số này đặc biệt phù hợp với bài toán nhận dạng biển số xe, bởi chỉ cần sai một ký tự cũng có thể dẫn đến nhận dạng sai hoàn toàn phương tiện.

Kết quả thực nghiệm cho thấy:

- Giá trị CER thấp nhất đạt 4.74%, tương ứng với độ chính xác ký tự cao.
- CER duy trì ổn định ở mức thấp trong quá trình đánh giá, phản ánh khả năng nhận dạng đáng tin cậy của mô hình trong điều kiện dữ liệu thực tế.

Character Accuracy: là chỉ số phản ánh tỷ lệ ký tự được nhận dạng đúng trên tổng số ký tự cần nhận dạng. Chỉ số này có mối quan hệ nghịch đảo với CER và giúp biểu diễn trực quan mức độ chính xác của mô hình OCR.

Trong báo cáo, với CER = 4.74%, mô hình EasyOCR đạt:

- Character Accuracy xấp xỉ 95.26%.

Mức độ chính xác này cho thấy mô hình đáp ứng tốt yêu cầu của hệ thống nhận dạng biển số xe trong bối cảnh quản lý bãi xe thông minh, nơi số lượng ký tự trên mỗi biển số là hạn chế và có cấu trúc rõ ràng.

Inference Time (Thời gian suy luận): Bên cạnh độ chính xác, hiệu năng xử lý cũng là một tiêu chí quan trọng khi đánh giá khả năng triển khai thực tế của mô hình OCR.

Kết quả đo đạc trong đề tài cho thấy:

- Thời gian suy luận trung bình của EasyOCR trên CPU dao động trong khoảng 300–350 ms cho mỗi ảnh biển số.

Mức thời gian này phù hợp với kịch bản xử lý theo từng lượt xe vào/ra trong bãi xe, đặc biệt khi hệ thống được triển khai kết hợp với mô hình phát hiện YOLOv8 và cơ chế xử lý tuần tự.

5.3.2.2. Phân tích các đường cong huấn luyện và đánh giá

Trong quá trình huấn luyện và đánh giá mô hình EasyOCR, các đường cong đặc trưng như Training Loss, Validation Loss và Character Error Rate (CER) được sử dụng nhằm theo dõi quá trình hội tụ và đánh giá khả năng tổng quát hóa của mô hình.

5.3.2.2.1. Đường cong Training Loss

Đường cong training loss cho thấy xu hướng giảm dần và hội tụ ổn định theo thời gian huấn luyện. Điều này phản ánh mô hình học được các đặc trưng hình thái và cấu trúc của ký tự trên biển số xe một cách hiệu quả, đồng thời không xuất hiện hiện tượng dao động lớn trong quá trình tối ưu.

5.3.2.2.2. Đường cong Validation Loss

Validation loss giảm và duy trì ổn định trong quá trình đánh giá, không xuất hiện xu hướng tăng trở lại ở các giai đoạn sau. Điều này cho thấy mô hình không gặp hiện tượng overfitting nghiêm trọng, dù vẫn tồn tại khoảng chênh lệch nhất định giữa training loss và validation loss.

5.3.2.2.3. Đường cong CER

Đường cong CER giảm rõ rệt trong giai đoạn đầu huấn luyện và sau đó duy trì ở mức thấp, phản ánh khả năng cải thiện nhanh chóng độ chính xác nhận dạng ký tự của

mô hình. Việc CER ổn định ở mức thấp cho thấy EasyOCR có khả năng duy trì hiệu suất tốt khi áp dụng trên dữ liệu chưa từng xuất hiện trong tập huấn luyện.

5.3.2.3. Đánh giá tổng quan

Tổng hợp các chỉ số đánh giá cho thấy mô hình EasyOCR đạt hiệu suất cao trong bài toán nhận dạng ký tự trên biển số xe:

- CER thấp (4.74%).
- Độ chính xác ký tự trên 95%.
- Thời gian suy luận phù hợp cho triển khai thực tế.

Mặc dù vẫn tồn tại hiện tượng nhầm lẫn giữa một số cặp ký tự có hình dạng tương đồng (như 8/B, 0/D, 4/9), các lỗi này đã được giảm thiểu đáng kể thông qua các bước tiền xử lý và hậu xử lý bằng Regex theo quy chuẩn biển số Việt Nam. Nhìn chung, EasyOCR đáp ứng tốt yêu cầu kỹ thuật của hệ thống ALPR trong bối cảnh ứng dụng ITS và sẵn sàng cho triển khai thực tế.

5.3.3. Metrics đánh giá hệ thống end-to-end

5.3.3.1. Thiết lập môi trường thử nghiệm

5.3.3.1.1. Cấu hình phần cứng và phần mềm

Hai môi trường thử nghiệm đã được cấu hình như sau:

- Môi trường 1: GPU-based Processing (trên Kaggle).
  - Hardware: GPU T4x2 (CUDA-enabled).
  - Framework: PyTorch với CUDA support.
  - Device: cuda.
  - Mục đích: Đánh giá hiệu năng suy luận tối ưu của mô hình trong điều kiện có tăng tốc phần cứng (GPU), làm cơ sở tham chiếu cho khả năng triển khai thực tế.
- Môi trường 2: CPU-based Processing.
  - Hardware: CPU only (no GPU acceleration).
  - Framework: PyTorch CPU version.
  - Device: cpu.
  - Mục đích: Đánh giá khả năng triển khai trên thiết bị không có GPU.

5.3.3.1.2. Dataset thử nghiệm

Bộ dữ liệu sử dụng cho quá trình đánh giá có các đặc điểm sau:

- Tổng số ảnh: 200 ảnh biển số đã được cắt và chuẩn bị sẵn.
- Nguồn: Dataset thử nghiệm đã được đánh nhãn (label) với dữ liệu thực tế (ground truth).
- Định dạng: Các ảnh đầu vào chỉ chứa vùng biển số, đã qua giai đoạn phát hiện (detection).
- Phân bố: Bao gồm các điều kiện ánh sáng và góc chụp đa dạng để đảm bảo tính tổng quát.

5.3.3.2. Phương pháp đánh giá

Hệ thống được đánh giá một cách toàn diện dựa trên hai nhóm chỉ số cốt lõi: độ chính xác của kết quả nhận dạng và hiệu năng xử lý. Các chỉ số này cung cấp một cái nhìn đầy đủ về cả chất lượng mô hình lẫn khả năng đáp ứng trong các ứng dụng thực tế. Thuật ngữ end-to-end được sử dụng để chỉ toàn bộ chuỗi xử lý từ ảnh biển số đã được phát hiện sẵn đến kết quả chuỗi ký tự đầu ra.

5.3.3.2.1. Độ chính xác (Accuracy)

- End-to-End Accuracy: Tỷ lệ phần trăm các biển số được nhận dạng chính xác hoàn toàn trên toàn bộ chuỗi ký tự.
- Character Error Rate (CER): Tỷ lệ lỗi tính trên từng ký tự riêng lẻ, cho phép đánh giá sâu hơn về các loại lỗi cụ thể.

5.3.3.2.2. Hiệu năng (Performance)

- Throughput: Thông lượng, được đo bằng số lượng ảnh hệ thống có thể xử lý trong một giây (images/s).
- Latency: Độ trễ, là thời gian xử lý trung bình cho mỗi ảnh, được đo bằng mili giây (ms).
- Percentile Latency: Độ trễ phân vị thứ 95 (P95), một chỉ số quan trọng để đánh giá hiệu năng trong trường hợp xấu nhất (worst-case performance).

Metric	Ký hiệu	Định nghĩa	Đơn vị
End-to-End Accuracy	Acc	Tỷ lệ ảnh biển số được nhận dạng đúng hoàn toàn chuỗi ký tự	%
Character Error Rate	CER	Số ký tự sai chia cho tổng số ký tự ground truth	%
Throughput	TPS	Số ảnh xử lý được trong 1 giây	images/s
Average Latency	L_avg	Thời gian xử lý trung bình cho một ảnh	ms
P95 Latency	L_p95	Thời gian xử lý mà 95% ảnh có latency nhỏ hơn	ms

Bảng 5.3: Định nghĩa các chỉ số đánh giá hệ thống

Tham số	Giá trị
Model	EasyOCR (fine-tuned)
Batch size	1
Precision	FP32
Decoding strategy	Greedy decoding
Max sequence length	theo cấu hình mặc định
Device	GPU / CPU

Bảng 5.4: Cấu hình suy luận mô hình EasyOCR trong quá trình đánh giá

5.3.3.2.3. Phân tích chi tiết từng giai đoạn

Quy trình xử lý (pipeline) được phân tích theo từng giai đoạn để xác định các điểm nghẽn về hiệu năng.

→ Input Image

→ Image Loading

→ Preprocessing

→ EasyOCR Preprocessing

→ EasyOCR Inference

→ Output Text

Giai đoạn phát hiện biển số (License Plate Detection) được thực hiện trước và nằm ngoài phạm vi đánh giá của mục này; do đó pipeline chỉ tập trung vào giai đoạn nhận dạng ký tự (recognition).

5.3.3.3. Kết quả đánh giá trên GPU

5.3.3.3.1. Độ chính xác tổng thể

Metric	Value	Percentage
Total images evaluated	200	100%
Recognition Success	194	97.0%
Recognition Fail	6	3.0%
End-to-End Accuracy	194/200	97.0%

Bảng 5.5: Kết quả End-to-End Accuracy trên GPU

Hệ thống đã chứng tỏ hiệu quả vượt trội, với độ chính xác tổng thể đạt 97.0% trên tập dữ liệu thử nghiệm. Con số này vượt xa mục tiêu ban đầu đề ra là ≥85%, khẳng định rằng mô hình EasyOCR đã được tinh chỉnh (fine-tune) rất hiệu quả và phù hợp với đặc thù của bộ dữ liệu biển số xe Việt Nam.

5.3.3.3.2. Phân tích hiệu năng từng giai đoạn

Processing Stage	Avg Time	Min	Max	Std Dev	P95
Image Loading	2ms	1ms	22ms	2.6ms	4ms
Preprocessing	11ms	1ms	52ms	8.1ms	26ms

EasyOCR Preprocessing	8ms	6ms	13ms	0.9ms	9ms
EasyOCR Inference	121ms	96ms	154ms	10.0ms	133ms

Bảng 5.6: Latency breakdown theo giai đoạn xử lý (GPU)

Phân tích:

- Image Loading (2ms trung bình): Giai đoạn đọc file ảnh từ đĩa. Thời gian dao động nhẹ do sự khác biệt về kích thước file và cơ chế cache hit/miss của hệ điều hành.
- Preprocessing (11ms trung bình): Bao gồm các tác vụ như thay đổi kích thước và chuẩn hóa giá trị pixel. Độ lệch chuẩn (8.1ms) và giá trị P95 (26ms) tương đối cao cho thấy thời gian xử lý phụ thuộc vào kích thước ảnh đầu vào.
- EasyOCR Preprocessing (8ms trung bình): Chuẩn bị dữ liệu đầu vào cho mô hình EasyOCR. Giai đoạn này rất ổn định (độ lệch chuẩn chỉ 0.9ms) vì tất cả ảnh đều được đưa về một kích thước cố định (384x384).
- EasyOCR Inference (121ms trung bình): Đây là giai đoạn thực thi suy luận của mô hình Transformer để nhận dạng ký tự. Giai đoạn này là nút thắt cổ chai (bottleneck) chính của toàn bộ hệ thống, chiếm tới 85% tổng thời gian xử lý. Tuy nhiên, thời gian suy luận rất ổn định, với độ lệch chuẩn chỉ 10.0ms (tương đương 8.3% so với giá trị trung bình), cho thấy hiệu năng nhất quán và có thể dự đoán được trên GPU.

Nhận xét: Phân tích trên cho thấy EasyOCR Inference là giai đoạn cần được ưu tiên tối ưu hóa để cải thiện thông lượng tổng thể của hệ thống. Các phương pháp tiềm năng bao gồm Model quantization (INT8), Batch processing (xử lý đồng thời nhiều ảnh), và Model distillation (sử dụng một mô hình nhỏ hơn, nhanh hơn).

5.3.3.3.3. Tổng hợp hiệu năng GPU

Metric	Value	Target
Total processing time	28.57 s	-
Throughput	7.00 images/s	≥5
Avg latency (per-image)	142.85 ms	<200ms
P95 latency (per-image)	164.67 ms*	<250ms

*P95 latency được tính toán dựa trên phân phối thời gian xử lý của 200 ảnh test.

Bảng 5.7: Performance metrics tổng hợp (GPU)

Phân tích:

- Throughput: 7.00 images/s: Hệ thống có khả năng xử lý 7 ảnh mỗi giây, tương đương với 420 ảnh/phút hoặc 25,200 ảnh/giờ. Mức hiệu năng này hoàn toàn đáp ứng yêu cầu của hầu hết các ứng dụng thực tế như quản lý bãi xe hay kiểm soát ra vào, và có thể được tăng cường thêm bằng cách áp dụng xử lý theo lô (batch processing).
- Average Latency: 142.85 ms: Độ trễ trung bình thấp hơn 24% so với mục tiêu (<200ms), đủ nhanh cho các ứng dụng yêu cầu phản hồi gần thời gian thực và cho phép xử lý các luồng video với độ trễ chấp nhận được.
- P95 Latency: 164.67 ms: 95% yêu cầu được xử lý trong vòng chưa đầy 165ms. Điều này cho thấy hiệu năng của hệ thống không chỉ nhanh mà còn rất ổn định, ngăn ngừa các hiện tượng trễ đột ngột có thể ảnh hưởng đến trải nghiệm người dùng trong các hệ thống thời gian thực.

5.3.3.4. Kết quả đánh giá trên CPU

Việc đánh giá trên môi trường chỉ có CPU nhằm mục đích xác định tính khả thi khi triển khai hệ thống trên các thiết bị phổ thông hoặc các máy chủ không được trang bị GPU chuyên dụng, giúp cân bằng giữa chi phí và hiệu năng.

5.3.3.4.1. Độ chính xác tổng thể

Metric	Value	Percentage
Total images evaluated	200	100%
Recognition Success	194	97.0%
Recognition Fail	6	3.0%
End-to-End Accuracy	194/200	97.0%

Bảng 5.8: Kết quả End-to-End Accuracy trên CPU

Nhận xét: Độ chính xác trên CPU là 97.0%, hoàn toàn giống hệt với kết quả trên GPU chứng tỏ rằng:

- Kết quả suy luận của mô hình có tính xác định (deterministic), không phụ thuộc vào nền tảng phần cứng.
- Không có vấn đề về độ chính xác số học khi chuyển đổi giữa GPU và CPU.
- Hệ thống có thể được triển khai trên môi trường chỉ có CPU mà không làm suy giảm độ chính xác nhận dạng.

5.3.3.4.2. Phân tích hiệu năng từng giai đoạn

Processing Stage	Avg Time	Min	Max	Std Dev	P95
Image Loading	2ms	1ms	25ms	2.1ms	2ms
Preprocessing	12ms	1ms	65ms	8.5ms	26ms
EasyOCR Preprocessing	8ms	6ms	13ms	1.0ms	10ms
EasyOCR Inference	307ms	266ms	385ms	19.2ms	337ms

Bảng 5.9: Latency breakdown theo giai đoạn xử lý (CPU)

Environment	Min CER (%)	Final CER (%)	Ghi chú
GPU	4.74	4.85	CER thấp và ổn định
CPU	4.74	4.85	Không khác biệt so với GPU

Bảng 5.10: So sánh Character Error Rate (CER) trên GPU và CPU

Stage	GPU (ms)	CPU (ms)	Slowdown	% of Total Time
Image Loading	2	2	1.0×	GPU: 1.4%, CPU: 0.6%
Preprocessing	11	12	1.1×	GPU: 7.7%, CPU: 3.7%
EasyOCR Preprocessing	8	8	1.0×	GPU: 5.6%, CPU: 2.4%
EasyOCR Inference	121	307	2.54×	GPU: 84.6%, CPU: 93.4%
Total	143	329	2.30×	-

Bảng 5.11: so sánh GPU vs CPU performance

Phân tích:

- Image Loading và Preprocessing: Hiệu năng của các giai đoạn này gần như không đổi giữa hai môi trường, vì đây là các tác vụ phụ thuộc vào I/O và tính toán đơn giản, không tận dụng được khả năng tăng tốc của GPU.
- EasyOCR Inference - Sự khác biệt chính: Đây là điểm khác biệt lớn nhất. Thời gian suy luận trên CPU là 307ms, chậm hơn 2.54 lần so với GPU

(121ms). Nguyên nhân là do bản chất của mô hình Transformer chứa rất nhiều phép nhân ma trận, một tác vụ mà GPU với hàng nghìn lõi xử lý song song thực hiện hiệu quả hơn rất nhiều so với CPU, đặc biệt là các phép toán Attention có độ phức tạp bậc $O(n^2)$ theo chiều dài chuỗi, vốn được GPU xử lý hiệu quả hơn nhờ khả năng song song hóa cao.

- Phương sai tăng trên CPU: Độ lệch chuẩn trên CPU (19.2ms) gần gấp đôi so với GPU (10.0ms). Điều này là do các yếu tố ở cấp độ hệ thống và phần cứng, bao gồm lập lịch CPU (CPU scheduling), chuyển đổi ngữ cảnh (context switching) với các tiến trình khác, và tỷ lệ cache miss cao hơn.

Nhận xét: Trên môi trường CPU, giai đoạn EasyOCR Inference chiếm tới 93.4% tổng thời gian xử lý, cho thấy đây là một nút thắt cổ chai cực kỳ nghiêm trọng và việc tối ưu hóa nó là bắt buộc nếu muốn triển khai trên các thiết bị không có GPU.

5.3.3.4.3. Tổng hợp hiệu năng CPU và so sánh GPU vs CPU

Metric	Value	Target
Total processing time	65.75 s	-
Throughput	3.04 images/s	≥2
Avg latency (per-image)	328.73 ms	<500ms
P95 latency (per-image)	367.18 ms	<600ms

Bảng 5.12: Performance metrics tổng hợp (CPU)

Phân tích comparative performance:

Metric	GPU	CPU	Ratio (CPU/GPU)
Throughput	7.00 img/s	3.04 img/s	0.43× (57% slower)
Avg Latency	142.85 ms	328.73 ms	2.30× slower
P95 Latency	164.67 ms	367.18 ms	2.23× slower
Total Time (200 imgs)	28.57 s	65.75 s	2.30× slower

Bảng 5.13: GPU vs CPU Performance Comparison

5.4. Hạn chế của hệ thống

Mặc dù hệ thống đạt được độ chính xác phù hợp với mục tiêu nghiên cứu (dựa trên đánh giá chung ở chương 1 và 4), qua quá trình thực nghiệm và kiểm thử, nhóm nghiên

cứu đã ghi nhận một số hạn chế cần được khắc phục để tối ưu hóa khả năng vận hành thực tế:

5.4.1. Độ chính xác và sai số mô hình

- Sự nhầm lẫn ký tự tương đồng: Mô hình EasyOCR vẫn gặp khó khăn trong việc phân biệt các cặp ký tự có đặc điểm hình thái giống nhau như: 4 và 9, 8 và B, 0 và D. Điều này chiếm khoảng 60% trong tổng số lỗi nhận diện (dựa trên hậu xử lý regex ở chương 4).
- Hiện tượng Overfitting nhẹ: Qua đánh giá huấn luyện (nếu có curve loss từ code ở chương 4), mô hình có thể quá khớp với dataset huấn luyện, dẫn đến hoạt động kém ổn định hơn khi gặp biển số có font chữ lạ hoặc điều kiện ánh sáng biến đổi.
- Nhạy cảm với nhiễu môi trường: Hệ thống giảm độ chính xác rõ rệt (xuống dưới 85%) trong các điều kiện ánh sáng cực yếu, sương mù dày đặc hoặc khi biển số bị che khuất cơ học bởi bùn đất quá 30% diện tích (khớp với tiền xử lý CLAHE/Otsu ở chương 4).

5.4.2. Hiệu năng tính toán (Performance)

- Độ trễ trên CPU (Latency): Thời gian xử lý trung bình trên CPU là khoảng 300-350 ms (dựa trên inference time từ code ở chương 4), tương đương khoảng 3 FPS. Tốc độ này chưa đáp ứng được yêu cầu giám sát các phương tiện di chuyển với vận tốc cao (> 40km/h) tại các trạm thu phí không dừng.
- Phụ thuộc vào tài nguyên phần cứng: Việc sử dụng EasyOCR (dựa trên PyTorch) đòi hỏi bộ nhớ GPU lớn, gây khó khăn cho việc triển khai trên các thiết bị Edge-AI có cấu hình thấp hoặc các camera thông minh tích hợp sẵn chip xử lý yếu.

5.5. Điều kiện triển khai thực tế

Để hệ thống ALPR đạt được hiệu quả tối ưu như trong điều kiện thử nghiệm (với YOLOv8 và EasyOCR), việc triển khai thực tế cần tuân thủ các yêu cầu kỹ thuật sau:

5.5.1. Yêu cầu về Hạ tầng phần cứng

- Đơn vị xử lý trung tâm: Khuyến nghị sử dụng GPU hỗ trợ nhân CUDA (như NVIDIA T4 x2 trên Kaggle ở chương 4) để đảm bảo độ trễ dưới 150ms.
- Hệ thống Camera:
  + Độ phân giải tối thiểu: HD (1280x720) để đảm bảo vùng biển số có kích thước ít nhất 100x50 pixels (khớp với capture frame ở chương 4).
  + Tốc độ màn trập (Shutter Speed): Cần thiết lập cao (> 1/1000s) để tránh hiện tượng nhòe chuyển động (Motion Blur).
- Kết nối mạng: Đảm bảo băng thông ổn định nếu triển khai theo mô hình Client-Server để truyền tải dữ liệu hình ảnh về Database (qua MQTT ở chương 3-4).

5.5.2. Môi trường và Góc lắp đặt

- Điều kiện ánh sáng: Cần lắp đặt thêm đèn LED hồng ngoại hỗ trợ cho camera vào ban đêm để đảm bảo cường độ sáng ổn định trên bề mặt biển số (hỗ trợ tiền xử lý ảnh ở chương 4).

- Góc quan sát: Góc nghiêng giữa trục camera và mặt phẳng biển số xe không được vượt quá 45 độ theo phương ngang và 30 độ theo phương thẳng đứng để giảm thiểu biến dạng phối cảnh.
- Khoảng cách: Tối ưu trong khoảng từ 3m đến 7m tùy thuộc vào tiêu cự ống kính camera.

5.6. Tiềm năng cải tiến

Dựa trên những hạn chế đã phân tích, nhóm đề xuất các hướng cải tiến trong tương lai nhằm nâng cao giá trị thực tiễn của dự án:

5.6.1. Tối ưu hóa mô hình và Thuật toán

- Kỹ thuật Nén mô hình (Model Compression): Áp dụng Quantization (chuyển đổi sang INT8) và Knowledge Distillation để giảm dung lượng mô hình EasyOCR, giúp tăng tốc độ xử lý trên CPU lên 2-3 lần mà không mất quá nhiều độ chính xác (tích hợp với ONNX/TensorRT ở chương 4).
- Tăng cường dữ liệu (Data Augmentation): Bổ sung thêm tập dữ liệu biên tập (synthetic data) mô phỏng các điều kiện thời tiết khắc nghiệt như mưa lớn, chói sáng và các biển số bị hỏng/mờ để tăng tính tổng quát hóa cho mô hình (dựa trên Roboflow ở chương 4).

5.6.2. Mở rộng chức năng hệ thống

- Nhận diện đa đặc điểm (Multi-attribute Recognition): Tích hợp thêm các module nhận diện màu sắc xe, hãng xe (Make/Model Recognition) và loại xe (xe tải, xe khách, xe con) để phục vụ công tác điều tra an ninh và phân luồng giao thông.
- Hệ thống Theo dõi (Tracking): Kết hợp thuật toán ByteTrack hoặc Sort để theo dõi hành trình của xe trong video, giúp loại bỏ việc nhận diện trùng lặp một phương tiện nhiều lần (mở rộng từ pipeline ở chương 4).
- Tích hợp Cloud và Mobile: Phát triển ứng dụng di động cho lực lượng chức năng để tra cứu nhanh thông tin biển số từ cơ sở dữ liệu tập trung thông qua Cloud API (liên kết với ReactJS ở chương 3).
- Thanh toán tự động: Liên kết với các cổng thanh toán điện tử (e-wallet) để hiện thực hóa quy trình thu phí đỗ xe tự động hoàn toàn (Touchless Parking, khớp với use case xe vào/ra ở chương 3).

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Summary essence của hệ thống ALPR

Mục này đóng vai trò "linh hồn" của chương kết luận, trả lời bản chất hệ thống là gì.

6.1.1. Bản chất và phạm vi của hệ thống

Hệ thống ALPR được xây dựng như một hệ thống phần mềm hoàn chỉnh, không phải chỉ là một mô hình học máy riêng lẻ.

Toàn bộ quy trình nhận diện biển số được tổ chức thành một pipeline end-to-end, bao gồm:

- Tiếp nhận dữ liệu đầu vào (ảnh/video từ camera).
- Xử lý và suy luận (phát hiện biển số bằng YOLOv8, nhận dạng ký tự bằng EasyOCR).
- Sinh kết quả đầu ra có thể sử dụng trực tiếp cho nghiệp vụ (lưu trữ vào MySQL, quản lý xe vào/ra).

Phạm vi hệ thống:

- Hướng tới bài toán nhận diện biển số xe trong bối cảnh giao thông thực tế tại Việt Nam (biển số ô tô và xe máy).
- Phù hợp với môi trường ITS thay vì chỉ phục vụ thử nghiệm học thuật, tập trung vào bãi đỗ xe thông minh, kiểm soát ra/vào.

6.1.2. Đặc trưng cốt lõi của hệ thống

Tự động hóa hoàn toàn:

- Giảm thiểu can thiệp thủ công nhờ pipeline tích hợp tiền xử lý (CLAHE, Otsu Thresholding), hậu xử lý (regex và confidence score >0.7).
- Giảm sai sót do con người thông qua logic quản lý xe (kiểm tra trạng thái trong DB).

Khả năng thích nghi:

- Hoạt động trong nhiều điều kiện môi trường và bối cảnh giao thông khác nhau (ánh sáng thay đổi, góc nghiêng).

Thiết kế định hướng ứng dụng:

- Ưu tiên tính ổn định (Docker Compose cho deployment), khả năng mở rộng (MQTT cho giao tiếp) và tích hợp (FastAPI cho API, ReactJS cho frontend).

6.1.3. Giá trị tổng quát và vai trò của hệ thống

Thể hiện một cách tiếp cận hiện đại đối với bài toán ALPR, kết hợp học sâu (YOLOv8 + EasyOCR) với hệ thống quản lý dữ liệu.

Đóng vai trò như:

- Một khối chức năng độc lập (module nhận diện và quản lý).

- Có thể tái sử dụng và tích hợp vào các hệ thống giao thông thông minh quy mô lớn hơn (như giám sát đô thị).

6.2. Tổng quan và mục tiêu của dự án

6.2.1. Bối cảnh xuất phát của đề tài

Sự gia tăng nhanh chóng của phương tiện giao thông tại Việt Nam, dẫn đến nhu cầu tự động hóa quản lý và giám sát.

Nhu cầu tự động hóa trong quản lý và giám sát giao thông, đặc biệt trong bối cảnh ùn tắc đô thị và bãi đỗ xe.

Vai trò ngày càng quan trọng của các hệ thống ITS trong đô thị hiện đại, nơi ALPR có thể hỗ trợ kiểm soát ra/vào và thu phí tự động.

6.2.2. Mục tiêu tổng thể của dự án

Xây dựng một hệ thống ALPR:

- Phù hợp với điều kiện giao thông tại Việt Nam (biển số chuẩn Việt Nam, điều kiện môi trường phức tạp).
- Có khả năng ứng dụng thực tế (bãi đỗ xe thông minh, giám sát giao thông).

Các mục tiêu cụ thể:

- Đảm bảo độ chính xác nhận diện (phù hợp với yêu cầu nghiên cứu, giảm nhầm lẫn ký tự qua hậu xử lý).
- Đáp ứng yêu cầu xử lý gần thời gian thực (inference time tối ưu trên GPU/CPU).
- Có khả năng mở rộng và tích hợp với các hệ thống khác (qua MQTT, Docker).

6.3. Tóm tắt cách tiếp cận và giải pháp đã thực hiện

6.3.1. Cách tiếp cận tổng thể

Tiếp cận bài toán ALPR theo hướng hệ thống hóa, xem ALPR là một chuỗi các bước xử lý liên kết chặt chẽ từ thu nhận dữ liệu đến lưu trữ và quản lý.

6.3.2. Các quyết định giải pháp chính

Áp dụng các mô hình học sâu hiện đại cho từng giai đoạn: YOLOv8 cho phát hiện biển số, EasyOCR cho nhận dạng ký tự, tránh các phương pháp thủ công dễ gây sai lệch.

Thiết kế pipeline thống nhất và nhất quán, với tiền xử lý (Grayscale, Blur, Thresholding) và hậu xử lý (regex chuẩn hóa định dạng biển số).

6.3.3. Định hướng triển khai

Hệ thống được xây dựng với mục tiêu:

- Không chỉ đạt kết quả tốt trong môi trường thử nghiệm (Kaggle GPU T4 x2).
- Mà còn có khả năng triển khai trong các bài toán thực tế của ITS (tích hợp barrier, dashboard thống kê).

6.4. Tổng hợp các kết quả chính đạt được

6.4.1. Kết quả ở mức hệ thống

Hệ thống vận hành ổn định trong nhiều bối cảnh khác nhau, đáp ứng các yêu cầu cơ bản của bài toán nhận diện biển số.

6.4.2. Đánh giá tổng thể

Độ chính xác tổng thể đáp ứng nhu cầu ứng dụng, hiệu năng xử lý phù hợp cho các hệ thống giám sát giao thông.

6.4.3. Giá trị thực nghiệm

Chứng minh tính khả thi của giải pháp đề xuất, xác nhận tính hợp lý của hướng tiếp cận end-to-end.

Kết quả chính	Mô tả	Ý nghĩa
Mô hình phát hiện biển số (YOLOv8)	Xác định chính xác vị trí biển số trong ảnh/video, xử lý nhiều xe đồng thời, góc nhìn đa dạng.	Đảm bảo độ ổn định cao cho pipeline, phù hợp ITS quy mô nhỏ/trung bình.
Nhận dạng ký tự (EasyOCR)	Nhận dạng chính xác ký tự, giảm nhầm lẫn qua allowlist và hậu xử lý regex.	Độ chính xác phù hợp, hỗ trợ lưu trữ và tra cứu biển số Việt Nam.
Pipeline end-to-end	Hoàn chỉnh từ thu nhận đến hậu xử lý, tự động hóa quy trình.	Dễ mở rộng, tích hợp với quản lý xe vào/ra.
Hệ thống quản lý dữ liệu (MySQL)	Lưu trữ thông tin xe, thời gian vào/ra, ảnh; hỗ trợ truy vấn và dashboard.	Ứng dụng thực tế cho bãi đỗ xe, giám sát giao thông.
Triển khai (Docker, MQTT)	Container hóa cho deployment, giao tiếp thời gian thực.	Tăng tính ổn định và khả năng tích hợp phần cứng (barrier).

Bảng 6.1: Tổng hợp kết quả chính đạt được

6.5. Giá trị khoa học và đóng góp của đề tài

6.5.1. Đóng góp về mặt học thuật

- Cung cấp một quy trình hoàn chỉnh cho bài toán ALPR, kết hợp YOLOv8 và EasyOCR với xử lý ảnh (OpenCV).
- Làm rõ khả năng áp dụng deep learning cho dữ liệu biển số Việt Nam (font chữ, định dạng đặc thù).
- Góp phần bổ sung tài liệu tham khảo cho lĩnh vực thị giác máy tính và ITS, với roadmap và timeline rõ ràng.

6.5.2. Đóng góp về mặt thực tiễn

- Hệ thống có thể làm nền tảng cho:

- Ứng dụng giao thông thông minh (bãi đỗ xe, kiểm soát ra/vào).
- Hệ thống giám sát và quản lý phương tiện (tra cứu lịch sử, thống kê).
- Có tiềm năng chuyển đổi từ nghiên cứu sang triển khai thực tế, với chi phí thấp (chạy trên CPU phổ thông).

6.6. Ý nghĩa của hệ thống đối với Hệ thống Giao thông Thông minh (ITS)

Đóng vai trò là một thành phần kỹ thuật quan trọng trong ITS, góp phần tự động hóa thu thập dữ liệu phương tiện.

Góp phần:

- Tự động hóa thu thập dữ liệu phương tiện, nâng cao hiệu quả quản lý giao thông (giảm ùn tắc tại bãi đỗ).
- Hỗ trợ ra quyết định trong điều hành đô thị (thống kê lưu lượng xe qua dashboard).

6.7. Những hạn chế còn tồn tại

Hạn chế về dữ liệu:

- Dữ liệu huấn luyện còn hạn chế về số lượng và độ đa dạng so với hệ thống thương mại, chưa bao phủ đầy đủ mọi kịch bản thực tế (như thời tiết khắc nghiệt).

Hạn chế về kỹ thuật:

- Hiệu năng phụ thuộc vào chất lượng dữ liệu đầu vào (ánh sáng yếu, occlusion >30%), dẫn đến độ chính xác giảm.
- Một số tình huống phức tạp vẫn là thách thức (nhầm lẫn ký tự tương đồng trong EasyOCR).

Hạn chế về triển khai:

- Chưa được kiểm chứng trong môi trường vận hành lâu dài hoặc quy mô lớn.
- Chưa đánh giá đầy đủ các yếu tố hạ tầng thực tế (bảo mật dữ liệu, quyền riêng tư).

6.8. Hướng phát triển và mở rộng trong tương lai

Mở rộng quy mô hệ thống (Scaling):

- Hỗ trợ nhiều camera và luồng dữ liệu lớn qua load balancing (Kubernetes).
- Phát triển kiến trúc xử lý phân tán để tăng throughput.

Phát triển theo hướng Edge Computing:

- Triển khai trên thiết bị biên (Raspberry Pi với ONNX) để giảm độ trễ.
- Tăng khả năng phản hồi thời gian thực mà không phụ thuộc server trung tâm.

Tích hợp với hệ sinh thái ITS (Integration):

- Kết nối với các hệ thống giao thông hiện có (barrier gate, camera IP qua RTSP).
- Đóng vai trò là mô-đun trong nền tảng đô thị thông minh (tích hợp thanh toán e-wallet).

Mở rộng và chuẩn hóa dữ liệu (Dataset):

- Thu thập dữ liệu đa dạng và quy mô lớn (synthetic data cho mưa, chói sáng).
- Chuẩn hóa để nâng cao chất lượng huấn luyện, sử dụng Roboflow để mở rộng augmentation.

Triển khai thực tế và thương mại hóa (Deployment):

- Hoàn thiện hệ thống cho môi trường vận hành (cloud như AWS, monitoring với Grafana).
- Đảm bảo ổn định, bảo mật và khả năng mở rộng (JWT authentication, backup DB).

6.9. Kết luận cuối cùng của đề tài

Khẳng định đề tài đã hoàn thành các mục tiêu đề ra, xây dựng được hệ thống ALPR có giá trị ứng dụng.

Nhấn mạnh tính thực tiễn (ứng dụng bãi đỗ xe, giám sát giao thông) và tiềm năng phát triển lâu dài trong ITS.